

Strings,Arrays,Function and Object

The JavaScript language

A large, bold, dark brown 'JS' logo is centered on the page. The background is a light orange color with a diagonal split into two shades of orange.

Functions

Quite often we need to perform a similar action in many places of the script.

For example, we need to show a nice-looking message when a visitor logs in, logs out and maybe somewhere else.

Functions are the main “building blocks” of the program. They allow the code to be called many times without repetition.

We’ve already seen examples of built-in functions, like `alert(message)`, `prompt(message, default)` and `confirm(question)`. But we can create functions of our own as well.

Function Declaration

To create a function we can use a *function declaration*.

It looks like this:

```
function showMessage() {  
  alert( 'Hello everyone!' );  
}
```

The `function` keyword goes first, then goes the *name of the function*, then a list of *parameters* between the parentheses (comma-separated, empty in the example above, we'll see examples later) and finally the code of the function, also named “the function body”, between curly braces.

```
function name(parameter1, parameter2, ... parameterN) {  
  // body  
}
```

Our new function can be called by its name: `showMessage()`.

For instance:

```
function showMessage() {  
  alert( 'Hello everyone!' );  
}  
  
showMessage();  
showMessage();
```

The call `showMessage()` executes the code of the function. Here we will see the message two times.

This example clearly demonstrates one of the main purposes of functions: to avoid code duplication.

If we ever need to change the message or the way it is shown, it's enough to modify the code in one place: the function which outputs it.

Local variables

A variable declared inside a function is only visible inside that function.

For example:

```
function showMessage() {  
  let message = "Hello, I'm JavaScript!"; // local variable  
  
  alert( message );  
}  
  
showMessage(); // Hello, I'm JavaScript!  
  
alert( message ); // <-- Error! The variable is local to the function
```

Outer variables

A function can access an outer variable as well, for example:

```
let userName = 'John';  
  
function showMessage() {  
  let message = 'Hello, ' + userName;  
  alert(message);  
}  
  
showMessage(); // Hello, John
```

The function has full access to the outer variable. It can modify it as well.

For instance:

```
let userName = 'John';

function showMessage() {
  userName = "Bob"; // (1) changed the outer variable

  let message = 'Hello, ' + userName;
  alert(message);
}

alert( userName ); // John before the function call

showMessage();

alert( userName ); // Bob, the value was modified by the function
```

The outer variable is only used if there's no local one.

If a same-named variable is declared inside the function then it *shadows* the outer one. For instance, in the code below the function uses the local `userName`. The outer one is ignored:

```
let userName = 'John';

function showMessage() {
  let userName = "Bob"; // declare a local variable

  let message = 'Hello, ' + userName; // Bob
  alert(message);
}

// the function will create and use its own userName
showMessage();

alert( userName ); // John, unchanged, the function did not access the outer variable
```

Global variables

Variables declared outside of any function, such as the outer `userName` in the code above, are called *global*.

Global variables are visible from any function (unless shadowed by locals).

It's a good practice to minimize the use of global variables. Modern code has few or no globals. Most variables reside in their functions. Sometimes though, they can be useful to store project-level data.

Parameters

We can pass arbitrary data to functions using parameters.

In the example below, the function has two parameters: `from` and `text`.

```
function showMessage(from, text) { // parameters: from, text
  alert(from + ': ' + text);
}

showMessage('Ann', 'Hello!'); // Ann: Hello! (*)
showMessage('Ann', "What's up?"); // Ann: What's up? (**)
```

When the function is called in lines `(*)` and `(**)`, the given values are copied to local variables `from` and `text`. Then the function uses them.

Here's one more example: we have a variable `from` and pass it to the function. Please note: the function changes `from`, but the change is not seen outside, because a function always gets a copy of the value:

```
function showMessage(from, text) {
  from = '*' + from + '*'; // make "from" look nicer

  alert( from + ': ' + text );
}

let from = "Ann";

showMessage(from, "Hello"); // *Ann*: Hello

// the value of "from" is the same, the function modified a local copy
alert( from ); // Ann
```

When a value is passed as a function parameter, it's also called an *argument*.

In other words, to put these terms straight:

- A parameter is the variable listed inside the parentheses in the function declaration (it's a declaration time term).
- An argument is the value that is passed to the function when it is called (it's a call time term).

We declare functions listing their parameters, then call them passing arguments.

In the example above, one might say: “the function `showMessage` is declared with two parameters, then called with two arguments: `from` and `"Hello"`”.

Default values

If a function is called, but an argument is not provided, then the corresponding value becomes `undefined`.

For instance, the aforementioned function `showMessage(from, text)` can be called with a single argument:

```
showMessage("Ann");
```

That's not an error. Such a call would output `"*Ann*: undefined"`. As the value for `text` isn't passed, it becomes `undefined`.

We can specify the so-called “default” (to use if omitted) value for a parameter in the function declaration, using `=`:

```
function showMessage(from, text = "no text given") {
  alert( from + ": " + text );
}

showMessage("Ann"); // Ann: no text given
```

Now if the `text` parameter is not passed, it will get the value `"no text given"`.

The default value also jumps in if the parameter exists, but strictly equals `undefined`, like this:

```
showMessage("Ann", undefined); // Ann: no text given
```

Here `"no text given"` is a string, but it can be a more complex expression, which is only evaluated and assigned if the parameter is missing. So, this is also possible:

```
function showMessage(from, text = anotherFunction()) {
  // anotherFunction() only executed if no text given
  // its result becomes the value of text
}
```

i Evaluation of default parameters

In JavaScript, a default parameter is evaluated every time the function is called without the respective parameter.

In the example above, `anotherFunction()` isn't called at all, if the `text` parameter is provided.

On the other hand, it's independently called every time when `text` is missing.

i Default parameters in old JavaScript code

Several years ago, JavaScript didn't support the syntax for default parameters. So people used other ways to specify them.

Nowadays, we can come across them in old scripts.

For example, an explicit check for `undefined`:

```
function showMessage(from, text) {  
  if (text === undefined) {  
    text = 'no text given';  
  }  
  
  alert( from + ": " + text );  
}
```

...Or using the `||` operator:

```
function showMessage(from, text) {  
  // If the value of text is falsy, assign the default value  
  // this assumes that text == "" is the same as no text at all  
  text = text || 'no text given';  
  ...  
}
```

Alternative default parameters

Sometimes it makes sense to assign default values for parameters at a later stage after the function declaration.

We can check if the parameter is passed during the function execution, by comparing it with `undefined`:

```
function showMessage(text) {  
  // ...  
  
  if (text === undefined) { // if the parameter is missing  
    text = 'empty message';  
  }  
  
  alert(text);  
}  
  
showMessage(); // empty message
```

...Or we could use the `||` operator:

```
function showMessage(text) {  
  // if text is undefined or otherwise falsy, set it to 'empty'  
  text = text || 'empty';  
  ...  
}
```

Modern JavaScript engines support the [nullish coalescing operator](#) `??`, it's better when most falsy values, such as `0`, should be considered "normal":

```
function showCount(count) {  
  // if count is undefined or null, show "unknown"  
  alert(count ?? "unknown");  
}
```

```
}

showCount(0); // 0
showCount(null); // unknown
showCount(); // unknown
```

Returning a value

A function can return a value back into the calling code as the result.

The simplest example would be a function that sums two values:

```
function sum(a, b) {
  return a + b;
}

let result = sum(1, 2);
alert( result ); // 3
```

The directive `return` can be in any place of the function. When the execution reaches it, the function stops, and the value is returned to the calling code (assigned to `result` above).

There may be many occurrences of `return` in a single function. For instance:

```
function checkAge(age) {
  if (age >= 18) {
    return true;
  } else {
    return confirm('Do you have permission from your parents?');
  }
}

let age = prompt('How old are you?', 18);

if ( checkAge(age) ) {
  alert( 'Access granted' );
} else {
  alert( 'Access denied' );
}
```

It is possible to use `return` without a value. That causes the function to exit immediately.

For example:

```
function showMovie(age) {
  if ( !checkAge(age) ) {
    return;
  }

  alert( "Showing you the movie" ); // (*)
  // ...
}
```

In the code above, if `checkAge(age)` returns `false`, then `showMovie` won't proceed to the `alert`.

A function with an empty `return` or without it returns `undefined`

If a function does not return a value, it is the same as if it returns `undefined`:

```
function doNothing() { /* empty */ }

alert( doNothing() === undefined ); // true
```

An empty `return` is also the same as `return undefined`:

```
function doNothing() {
  return;
}

alert( doNothing() === undefined ); // true
```

Never add a newline between `return` and the value

For a long expression in `return`, it might be tempting to put it on a separate line, like this:

```
return
(some + long + expression + or + whatever * f(a) + f(b))
```

That doesn't work, because JavaScript assumes a semicolon after `return`. That'll work the same as:

```
return;
(some + long + expression + or + whatever * f(a) + f(b))
```

So, it effectively becomes an empty return.

If we want the returned expression to wrap across multiple lines, we should start it at the same line as `return`. Or at least put the opening parentheses there as follows:

```
return (
  some + long + expression
  + or +
  whatever * f(a) + f(b)
)
```

And it will work just as we expect it to.

Naming a function

Functions are actions. So their name is usually a verb. It should be brief, as accurate as possible and describe what the function does, so that someone reading the code gets an indication of what the function does.

It is a widespread practice to start a function with a verbal prefix which vaguely describes the action. There must be an agreement within the team on the meaning of the prefixes.

For instance, functions that start with `"show"` usually show something.

Function starting with...

- `"get..."` – return a value,
- `"calc..."` – calculate something,
- `"create..."` – create something,
- `"check..."` – check something and return a boolean, etc.

Examples of such names:

```

showMessage(...) // shows a message
getAge(...)      // returns the age (gets it somehow)
calcSum(...)     // calculates a sum and returns the result
createForm(...)  // creates a form (and usually returns it)
checkPermission(...) // checks a permission, returns true/false

```

With prefixes in place, a glance at a function name gives an understanding what kind of work it does and what kind of value it returns.

i One function – one action

A function should do exactly what is suggested by its name, no more.

Two independent actions usually deserve two functions, even if they are usually called together (in that case we can make a 3rd function that calls those two).

A few examples of breaking this rule:

- `getAge` – would be bad if it shows an `alert` with the age (should only get).
- `createForm` – would be bad if it modifies the document, adding a form to it (should only create it and return).
- `checkPermission` – would be bad if it displays the `access granted/denied` message (should only perform the check and return the result).

These examples assume common meanings of prefixes. You and your team are free to agree on other meanings, but usually they're not much different. In any case, you should have a firm understanding of what a prefix means, what a prefixed function can and cannot do. All same-prefixed functions should obey the rules. And the team should share the knowledge.

i Ultrashort function names

Functions that are used *very often* sometimes have ultrashort names.

For example, the [jQuery](#) framework defines a function with `$`. The [Lodash](#) library has its core function named `_`.

These are exceptions. Generally function names should be concise and descriptive.

Functions == Comments

Functions should be short and do exactly one thing. If that thing is big, maybe it's worth it to split the function into a few smaller functions. Sometimes following this rule may not be that easy, but it's definitely a good thing.

A separate function is not only easier to test and debug – its very existence is a great comment!

For instance, compare the two functions `showPrimes(n)` below. Each one outputs [prime numbers](#) up to `n`.

The first variant uses a label:

```

function showPrimes(n) {
  nextPrime: for (let i = 2; i < n; i++) {

    for (let j = 2; j < i; j++) {
      if (i % j == 0) continue nextPrime;
    }

    alert( i ); // a prime
  }
}

```

The second variant uses an additional function `isPrime(n)` to test for primality:

```

function showPrimes(n) {

  for (let i = 2; i < n; i++) {
    if (!isPrime(i)) continue;

    alert(i); // a prime
  }
}

```

```

}

function isPrime(n) {
  for (let i = 2; i < n; i++) {
    if ( n % i == 0) return false;
  }
  return true;
}

```

The second variant is easier to understand, isn't it? Instead of the code piece we see a name of the action (`isPrime`). Sometimes people refer to such code as *self-describing*.

So, functions can be created even if we don't intend to reuse them. They structure the code and make it readable.

Summary

A function declaration looks like this:

```

function name(parameters, delimited, by, comma) {
  /* code */
}

```

- Values passed to a function as parameters are copied to its local variables.
- A function may access outer variables. But it works only from inside out. The code outside of the function doesn't see its local variables.
- A function can return a value. If it doesn't, then its result is `undefined`.

To make the code clean and easy to understand, it's recommended to use mainly local variables and parameters in the function, not outer variables.

It is always easier to understand a function which gets parameters, works with them and returns a result than a function which gets no parameters, but modifies outer variables as a side effect.

Function naming:

- A name should clearly describe what the function does. When we see a function call in the code, a good name instantly gives us an understanding what it does and returns.
- A function is an action, so function names are usually verbal.
- There exist many well-known function prefixes like `create...`, `show...`, `get...`, `check...` and so on. Use them to hint what a function does.

Functions are the main building blocks of scripts. Now we've covered the basics, so we actually can start creating and using them. But that's only the beginning of the path. We are going to return to them many times, going more deeply into their advanced features.

✓ Tasks

Is "else" required?

importance: 4

The following function returns `true` if the parameter `age` is greater than `18`.

Otherwise it asks for a confirmation and returns its result:

```

function checkAge(age) {
  if (age > 18) {
    return true;
  } else {
    // ...
    return confirm('Did parents allow you?');
  }
}

```

Will the function work differently if `else` is removed?

```
function checkAge(age) {
  if (age > 18) {
    return true;
  }
  // ...
  return confirm('Did parents allow you?');
}
```

Is there any difference in the behavior of these two variants?

[To solution](#)

Rewrite the function using '?' or '||'

importance: 4

The following function returns `true` if the parameter `age` is greater than `18`.

Otherwise it asks for a confirmation and returns its result.

```
function checkAge(age) {
  if (age > 18) {
    return true;
  } else {
    return confirm('Did parents allow you?');
  }
}
```

Rewrite it, to perform the same, but without `if`, in a single line.

Make two variants of `checkAge`:

1. Using a question mark operator `?`
2. Using OR `||`

[To solution](#)

Function min(a, b)

importance: 1

Write a function `min(a, b)` which returns the least of two numbers `a` and `b`.

For instance:

```
min(2, 5) == 2
min(3, -1) == -1
min(1, 1) == 1
```

[To solution](#)

Function pow(x,n)

importance: 4

Write a function `pow(x, n)` that returns `x` in power `n`. Or, in other words, multiplies `x` by itself `n` times and returns the result.

```
pow(3, 2) = 3 * 3 = 9
pow(3, 3) = 3 * 3 * 3 = 27
pow(1, 100) = 1 * 1 * ... * 1 = 1
```

Create a web-page that prompts for `x` and `n`, and then shows the result of `pow(x, n)`.

[Run the demo](#)

P.S. In this task the function should support only natural values of `n` : integers up from `1` .

[To solution](#)

Function expressions

In JavaScript, a function is not a “magical language structure”, but a special kind of value.

The syntax that we used before is called a *Function Declaration*:

```
function sayHi() {  
  alert( "Hello" );  
}
```

There is another syntax for creating a function that is called a *Function Expression*.

It allows us to create a new function in the middle of any expression.

For example:

```
let sayHi = function() {  
  alert( "Hello" );  
};
```

Here we can see a variable `sayHi` getting a value, the new function, created as `function() { alert("Hello"); }`.

As the function creation happens in the context of the assignment expression (to the right side of `=`), this is a *Function Expression*.

Please note, there's no name after the `function` keyword. Omitting a name is allowed for Function Expressions.

Here we immediately assign it to the variable, so the meaning of these code samples is the same: “create a function and put it into the variable `sayHi`”.

In more advanced situations, that we'll come across later, a function may be created and immediately called or scheduled for a later execution, not stored anywhere, thus remaining anonymous.

Function is a value

Let's reiterate: no matter how the function is created, a function is a value. Both examples above store a function in the `sayHi` variable.

We can even print out that value using `alert`:

```
function sayHi() {  
  alert( "Hello" );  
}  
  
alert( sayHi ); // shows the function code
```

Please note that the last line does not run the function, because there are no parentheses after `sayHi`. There are programming languages where any mention of a function name causes its execution, but JavaScript is not like that.

In JavaScript, a function is a value, so we can deal with it as a value. The code above shows its string representation, which is the source code.

Surely, a function is a special value, in the sense that we can call it like `sayHi()`.

But it's still a value. So we can work with it like with other kinds of values.

We can copy a function to another variable:

```
function sayHi() { // (1) create  
  alert( "Hello" );  
}
```

```
let func = sayHi;    // (2) copy

func(); // Hello    // (3) run the copy (it works)!
sayHi(); // Hello    // this still works too (why wouldn't it)
```

Here's what happens above in detail:

1. The Function Declaration (1) creates the function and puts it into the variable named `sayHi`.
2. Line (2) copies it into the variable `func`. Please note again: there are no parentheses after `sayHi`. If there were, then `func = sayHi()` would write *the result of the call* `sayHi()` into `func`, not *the function* `sayHi` itself.
3. Now the function can be called as both `sayHi()` and `func()`.

We could also have used a Function Expression to declare `sayHi`, in the first line:

```
let sayHi = function() { // (1) create
  alert( "Hello" );
};

let func = sayHi; // (2)
// ...
```

Everything would work the same.

Why is there a semicolon at the end?

You might wonder, why do Function Expressions have a semicolon `;` at the end, but Function Declarations do not:

```
function sayHi() {
  // ...
}

let sayHi = function() {
  // ...
};
```

The answer is simple: a Function Expression is created here as `function(...) {...}` inside the assignment statement: `let sayHi = ...;`. The semicolon `;` is recommended at the end of the statement, it's not a part of the function syntax.

The semicolon would be there for a simpler assignment, such as `let sayHi = 5;`, and it's also there for a function assignment.

Callback functions

Let's look at more examples of passing functions as values and using function expressions.

We'll write a function `ask(question, yes, no)` with three parameters:

question

Text of the question

yes

Function to run if the answer is "Yes"

no

Function to run if the answer is "No"

The function should ask the `question` and, depending on the user's answer, call `yes()` or `no()`:

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

function showOk() {
```

```

    alert( "You agreed." );
}

function showCancel() {
    alert( "You canceled the execution." );
}

// usage: functions showOk, showCancel are passed as arguments to ask
ask("Do you agree?", showOk, showCancel);

```

In practice, such functions are quite useful. The major difference between a real-life `ask` and the example above is that real-life functions use more complex ways to interact with the user than a simple `confirm`. In the browser, such functions usually draw a nice-looking question window. But that's another story.

The arguments `showOk` and `showCancel` of `ask` are called **callback functions** or just **callbacks**.

The idea is that we pass a function and expect it to be “called back” later if necessary. In our case, `showOk` becomes the callback for “yes” answer, and `showCancel` for “no” answer.

We can use Function Expressions to write an equivalent, shorter function:

```

function ask(question, yes, no) {
    if (confirm(question)) yes()
    else no();
}

ask(
    "Do you agree?",
    function() { alert("You agreed."); },
    function() { alert("You canceled the execution."); }
);

```

Here, functions are declared right inside the `ask(...)` call. They have no name, and so are called *anonymous*. Such functions are not accessible outside of `ask` (because they are not assigned to variables), but that's just what we want here.

Such code appears in our scripts very naturally, it's in the spirit of JavaScript.

i A function is a value representing an “action”

Regular values like strings or numbers represent the *data*.

A function can be perceived as an *action*.

We can pass it between variables and run when we want.

Function Expression vs Function Declaration

Let's formulate the key differences between Function Declarations and Expressions.

First, the syntax: how to differentiate between them in the code.

- *Function Declaration*: a function, declared as a separate statement, in the main code flow:

```

// Function Declaration
function sum(a, b) {
    return a + b;
}

```

- *Function Expression*: a function, created inside an expression or inside another syntax construct. Here, the function is created on the right side of the “assignment expression” `=`:

```

// Function Expression
let sum = function(a, b) {
    return a + b;
};

```

The more subtle difference is *when* a function is created by the JavaScript engine.

A Function Expression is created when the execution reaches it and is usable only from that moment.

Once the execution flow passes to the right side of the assignment `let sum = function...` – here we go, the function is created and can be used (assigned, called, etc.) from now on.

Function Declarations are different.

A Function Declaration can be called earlier than it is defined.

For example, a global Function Declaration is visible in the whole script, no matter where it is.

That's due to internal algorithms. When JavaScript prepares to run the script, it first looks for global Function Declarations in it and creates the functions. We can think of it as an “initialization stage”.

And after all Function Declarations are processed, the code is executed. So it has access to these functions.

For example, this works:

```
sayHi("John"); // Hello, John

function sayHi(name) {
  alert( `Hello, ${name}` );
}
```

The Function Declaration `sayHi` is created when JavaScript is preparing to start the script and is visible everywhere in it.

...If it were a Function Expression, then it wouldn't work:

```
sayHi("John"); // error!

let sayHi = function(name) { // (*) no magic any more
  alert( `Hello, ${name}` );
};
```

Function Expressions are created when the execution reaches them. That would happen only in the line `(*)`. Too late.

Another special feature of Function Declarations is their block scope.

In strict mode, when a Function Declaration is within a code block, it's visible everywhere inside that block. But not outside of it.

For instance, let's imagine that we need to declare a function `welcome()` depending on the `age` variable that we get during runtime. And then we plan to use it some time later.

If we use Function Declaration, it won't work as intended:

```
let age = prompt("What is your age?", 18);

// conditionally declare a function
if (age < 18) {

  function welcome() {
    alert("Hello!");
  }

} else {

  function welcome() {
    alert("Greetings!");
  }

}

// ...use it later
welcome(); // Error: welcome is not defined
```

That's because a Function Declaration is only visible inside the code block in which it resides.

Here's another example:

```

let age = 16; // take 16 as an example

if (age < 18) {
  welcome(); // \ (runs)
             // |
  function welcome() { // |
    alert("Hello!"); // | Function Declaration is available
  }                // | everywhere in the block where it's declared
                  // |
  welcome(); // / (runs)
} else {

  function welcome() {
    alert("Greetings!");
  }
}

// Here we're out of curly braces,
// so we can not see Function Declarations made inside of them.

welcome(); // Error: welcome is not defined

```

What can we do to make `welcome` visible outside of `if`?

The correct approach would be to use a Function Expression and assign `welcome` to the variable that is declared outside of `if` and has the proper visibility.

This code works as intended:

```

let age = prompt("What is your age?", 18);

let welcome;

if (age < 18) {

  welcome = function() {
    alert("Hello!");
  };

} else {

  welcome = function() {
    alert("Greetings!");
  };

}

welcome(); // ok now

```

Or we could simplify it even further using a question mark operator `?:`:

```

let age = prompt("What is your age?", 18);

let welcome = (age < 18) ?
  function() { alert("Hello!"); } :
  function() { alert("Greetings!"); };

welcome(); // ok now

```

i When to choose Function Declaration versus Function Expression?

As a rule of thumb, when we need to declare a function, the first thing to consider is Function Declaration syntax. It gives more freedom in how to organize our code, because we can call such functions before they are declared.

That's also better for readability, as it's easier to look up `function f(...) {...}` in the code than `let f = function(...) {...};`. Function Declarations are more "eye-catching".

...But if a Function Declaration does not suit us for some reason, or we need a conditional declaration (we've just seen an example), then Function Expression should be used.

Summary

- Functions are values. They can be assigned, copied or declared in any place of the code.
- If the function is declared as a separate statement in the main code flow, that's called a "Function Declaration".
- If the function is created as a part of an expression, it's called a "Function Expression".
- Function Declarations are processed before the code block is executed. They are visible everywhere in the block.
- Function Expressions are created when the execution flow reaches them.

In most cases when we need to declare a function, a Function Declaration is preferable, because it is visible prior to the declaration itself. That gives us more flexibility in code organization, and is usually more readable.

So we should use a Function Expression only when a Function Declaration is not fit for the task. We've seen a couple of examples of that in this chapter, and will see more in the future.

Arrow functions, the basics

There's another very simple and concise syntax for creating functions, that's often better than Function Expressions.

It's called "arrow functions", because it looks like this:

```
let func = (arg1, arg2, ..., argN) => expression;
```

This creates a function `func` that accepts arguments `arg1..argN`, then evaluates the `expression` on the right side with their use and returns its result.

In other words, it's the shorter version of:

```
let func = function(arg1, arg2, ..., argN) {  
  return expression;  
};
```

Let's see a concrete example:

```
let sum = (a, b) => a + b;  
  
/* This arrow function is a shorter form of:  
  
let sum = function(a, b) {  
  return a + b;  
};  
*/  
  
alert( sum(1, 2) ); // 3
```

As you can see, `(a, b) => a + b` means a function that accepts two arguments named `a` and `b`. Upon the execution, it evaluates the expression `a + b` and returns the result.

- If we have only one argument, then parentheses around parameters can be omitted, making that even shorter.

For example:

```
let double = n => n * 2;  
// roughly the same as: let double = function(n) { return n * 2 }  
  
alert( double(3) ); // 6
```

- If there are no arguments, parentheses are empty, but they must be present:

```
let sayHi = () => alert("Hello!");  
  
sayHi();
```

Arrow functions can be used in the same way as Function Expressions.

For instance, to dynamically create a function:

```
let age = prompt("What is your age?", 18);

let welcome = (age < 18) ?
  () => alert('Hello!') :
  () => alert("Greetings!");

welcome();
```

Arrow functions may appear unfamiliar and not very readable at first, but that quickly changes as the eyes get used to the structure.

They are very convenient for simple one-line actions, when we're just too lazy to write many words.

Multiline arrow functions

The arrow functions that we've seen so far were very simple. They took arguments from the left of `=>`, evaluated and returned the right-side expression with them.

Sometimes we need a more complex function, with multiple expressions and statements. In that case, we can enclose them in curly braces. The major difference is that curly braces require a `return` within them to return a value (just like a regular function does).

Like this:

```
let sum = (a, b) => { // the curly brace opens a multiline function
  let result = a + b;
  return result; // if we use curly braces, then we need an explicit "return"
};

alert( sum(1, 2) ); // 3
```

More to come

Here we praised arrow functions for brevity. But that's not all!

Arrow functions have other interesting features.

To study them in-depth, we first need to get to know some other aspects of JavaScript, so we'll return to arrow functions later in the chapter [Arrow functions revisited](#).

For now, we can already use arrow functions for one-line actions and callbacks.

Summary

Arrow functions are handy for simple actions, especially for one-liners. They come in two flavors:

1. Without curly braces: `(...args) => expression` – the right side is an expression: the function evaluates it and returns the result. Parentheses can be omitted, if there's only a single argument, e.g. `n => n*2`.
2. With curly braces: `(...args) => { body }` – brackets allow us to write multiple statements inside the function, but we need an explicit `return` to return something.

Tasks

Rewrite with arrow functions

Replace Function Expressions with arrow functions in the code below:

```
function ask(question, yes, no) {
  if (confirm(question)) yes();
  else no();
}

ask(
  "Do you agree?",
```

```
result *= x;
assert.equal(pow(x, 2), result);

result *= x;
assert.equal(pow(x, 3), result);
});
```

P.S. Syntactically the test is correct and passes.

[To solution](#)

Polyfills and transpilers

The JavaScript language steadily evolves. New proposals to the language appear regularly, they are analyzed and, if considered worthy, are appended to the list at <https://tc39.github.io/ecma262/> and then progress to the [specification](#).

Teams behind JavaScript engines have their own ideas about what to implement first. They may decide to implement proposals that are in draft and postpone things that are already in the spec, because they are less interesting or just harder to do.

So it's quite common for an engine to implement only part of the standard.

A good page to see the current state of support for language features is <https://compat-table.github.io/compat-table/es6/> (it's big, we have a lot to study yet).

As programmers, we'd like to use most recent features. The more good stuff – the better!

On the other hand, how to make our modern code work on older engines that don't understand recent features yet?

There are two tools for that:

1. Transpilers.
2. Polyfills.

Here, in this chapter, our purpose is to get the gist of how they work, and their place in web development.

Transpilers

A [transpiler](#) is a special piece of software that translates source code to another source code. It can parse ("read and understand") modern code and rewrite it using older syntax constructs, so that it'll also work in outdated engines.

E.g. JavaScript before year 2020 didn't have the "nullish coalescing operator" `??`. So, if a visitor uses an outdated browser, it may fail to understand the code like `height = height ?? 100`.

A transpiler would analyze our code and rewrite `height ?? 100` into `(height !== undefined && height !== null) ? height : 100`.

```
// before running the transpiler
height = height ?? 100;

// after running the transpiler
height = (height !== undefined && height !== null) ? height : 100;
```

Now the rewritten code is suitable for older JavaScript engines.

Usually, a developer runs the transpiler on their own computer, and then deploys the transpiled code to the server.

Speaking of names, [Babel](#) is one of the most prominent transpilers out there.

Modern project build systems, such as [webpack](#), provide a means to run a transpiler automatically on every code change, so it's very easy to integrate into the development process.

Polyfills

New language features may include not only syntax constructs and operators, but also built-in functions.

For example, `Math.trunc(n)` is a function that “cuts off” the decimal part of a number, e.g `Math.trunc(1.23)` returns `1`.

In some (very outdated) JavaScript engines, there's no `Math.trunc`, so such code will fail.

As we're talking about new functions, not syntax changes, there's no need to transpile anything here. We just need to declare the missing function.

A script that updates/adds new functions is called “polyfill”. It “fills in” the gap and adds missing implementations.

For this particular case, the polyfill for `Math.trunc` is a script that implements it, like this:

```
if (!Math.trunc) { // if no such function
  // implement it
  Math.trunc = function(number) {
    // Math.ceil and Math.floor exist even in ancient JavaScript engines
    // they are covered later in the tutorial
    return number < 0 ? Math.ceil(number) : Math.floor(number);
  };
}
```

JavaScript is a highly dynamic language. Scripts may add/modify any function, even built-in ones.

One interesting polyfill library is [core-js](#), which supports a wide range of features and allows you to include only the ones you need.

Summary

In this chapter we'd like to motivate you to study modern and even “bleeding-edge” language features, even if they aren't yet well-supported by JavaScript engines.

Just don't forget to use a transpiler (if using modern syntax or operators) and polyfills (to add functions that may be missing). They'll ensure that the code works.

For example, later when you're familiar with JavaScript, you can setup a code build system based on [webpack](#) with the [babel-loader](#) plugin.

Good resources that show the current state of support for various features:

- <https://compat-table.github.io/compat-table/es6/> – for pure JavaScript.
- <https://caniuse.com/> – for browser-related functions.

P.S. Google Chrome is usually the most up-to-date with language features, try it if a tutorial demo fails. Most tutorial demos work with any modern browser though.

Objects: the basics

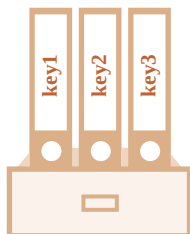
Objects

As we know from the chapter [Data types](#), there are eight data types in JavaScript. Seven of them are called “primitive”, because their values contain only a single thing (be it a string or a number or whatever).

In contrast, objects are used to store keyed collections of various data and more complex entities. In JavaScript, objects penetrate almost every aspect of the language. So we must understand them first before going in-depth anywhere else.

An object can be created with figure brackets `{...}` with an optional list of *properties*. A property is a “key: value” pair, where `key` is a string (also called a “property name”), and `value` can be anything.

We can imagine an object as a cabinet with signed files. Every piece of data is stored in its file by the key. It's easy to find a file by its name or add/remove a file.



An empty object (“empty cabinet”) can be created using one of two syntaxes:

```
let user = new Object(); // "object constructor" syntax
let user = {}; // "object literal" syntax
```



Usually, the figure brackets `{ ... }` are used. That declaration is called an *object literal*.

Literals and properties

We can immediately put some properties into `{ ... }` as “key: value” pairs:

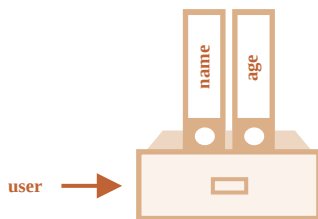
```
let user = { // an object
  name: "John", // by key "name" store value "John"
  age: 30 // by key "age" store value 30
};
```

A property has a key (also known as “name” or “identifier”) before the colon `:` and a value to the right of it.

In the `user` object, there are two properties:

1. The first property has the name `"name"` and the value `"John"`.
2. The second one has the name `"age"` and the value `30`.

The resulting `user` object can be imagined as a cabinet with two signed files labeled “name” and “age”.



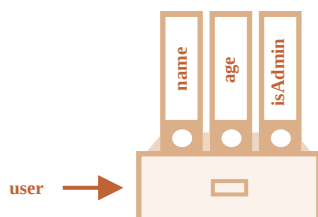
We can add, remove and read files from it at any time.

Property values are accessible using the dot notation:

```
// get property values of the object:
alert( user.name ); // John
alert( user.age ); // 30
```

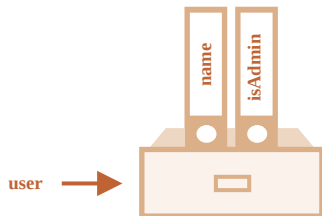
The value can be of any type. Let’s add a boolean one:

```
user.isAdmin = true;
```



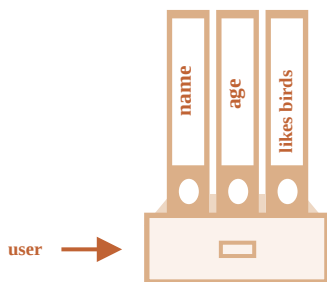
To remove a property, we can use the `delete` operator:

```
delete user.age;
```



We can also use multiword property names, but then they must be quoted:

```
let user = {  
  name: "John",  
  age: 30,  
  "likes birds": true // multiword property name must be quoted  
};
```



The last property in the list may end with a comma:

```
let user = {  
  name: "John",  
  age: 30,  
}
```

That is called a “trailing” or “hanging” comma. Makes it easier to add/remove/move around properties, because all lines become alike.

Square brackets

For multiword properties, the dot access doesn't work:

```
// this would give a syntax error  
user.likes birds = true
```

JavaScript doesn't understand that. It thinks that we address `user.likes`, and then gives a syntax error when comes across unexpected `birds`.

The dot requires the key to be a valid variable identifier. That implies: contains no spaces, doesn't start with a digit and doesn't include special characters (`$` and `_` are allowed).

There's an alternative “square bracket notation” that works with any string:

```
let user = {};  
  
// set  
user["likes birds"] = true;  
  
// get  
alert(user["likes birds"]); // true  
  
// delete  
delete user["likes birds"];
```

Now everything is fine. Please note that the string inside the brackets is properly quoted (any type of quotes will do).

Square brackets also provide a way to obtain the property name as the result of any expression – as opposed to a literal string – like from a variable as follows:

```
let key = "likes birds";

// same as user["likes birds"] = true;
user[key] = true;
```

Here, the variable `key` may be calculated at run-time or depend on the user input. And then we use it to access the property. That gives us a great deal of flexibility.

For instance:

```
let user = {
  name: "John",
  age: 30
};

let key = prompt("What do you want to know about the user?", "name");

// access by variable
alert( user[key] ); // John (if enter "name")
```

The dot notation cannot be used in a similar way:

```
let user = {
  name: "John",
  age: 30
};

let key = "name";
alert( user.key ) // undefined
```

Computed properties

We can use square brackets in an object literal, when creating an object. That's called *computed properties*.

For instance:

```
let fruit = prompt("Which fruit to buy?", "apple");

let bag = {
  [fruit]: 5, // the name of the property is taken from the variable fruit
};

alert( bag.apple ); // 5 if fruit="apple"
```

The meaning of a computed property is simple: `[fruit]` means that the property name should be taken from `fruit`.

So, if a visitor enters `"apple"`, `bag` will become `{apple: 5}`.

Essentially, that works the same as:

```
let fruit = prompt("Which fruit to buy?", "apple");
let bag = {};

// take property name from the fruit variable
bag[fruit] = 5;
```

...But looks nicer.

We can use more complex expressions inside square brackets:

```
let fruit = 'apple';
let bag = {
```

```
[fruit + 'Computers']: 5 // bag.appleComputers = 5
};
```

Square brackets are much more powerful than dot notation. They allow any property names and variables. But they are also more cumbersome to write.

So most of the time, when property names are known and simple, the dot is used. And if we need something more complex, then we switch to square brackets.

Property value shorthand

In real code, we often use existing variables as values for property names.

For instance:

```
function makeUser(name, age) {
  return {
    name: name,
    age: age,
    // ...other properties
  };
}

let user = makeUser("John", 30);
alert(user.name); // John
```

In the example above, properties have the same names as variables. The use-case of making a property from a variable is so common, that there's a special *property value shorthand* to make it shorter.

Instead of `name: name` we can just write `name`, like this:

```
function makeUser(name, age) {
  return {
    name, // same as name: name
    age,  // same as age: age
    // ...
  };
}
```

We can use both normal properties and shorthands in the same object:

```
let user = {
  name, // same as name: name
  age: 30
};
```

Property names limitations

As we already know, a variable cannot have a name equal to one of the language-reserved words like “for”, “let”, “return” etc.

But for an object property, there's no such restriction:

```
// these properties are all right
let obj = {
  for: 1,
  let: 2,
  return: 3
};

alert( obj.for + obj.let + obj.return ); // 6
```

In short, there are no limitations on property names. They can be any strings or symbols (a special type for identifiers, to be covered later).

Other types are automatically converted to strings.

For instance, a number `0` becomes a string `"0"` when used as a property key:

```
let obj = {
  0: "test" // same as "0": "test"
};

// both alerts access the same property (the number 0 is converted to string "0")
alert( obj["0"] ); // test
alert( obj[0] ); // test (same property)
```

There's a minor gotcha with a special property named `__proto__`. We can't set it to a non-object value:

```
let obj = {};
obj.__proto__ = 5; // assign a number
alert(obj.__proto__); // [object Object] - the value is an object, didn't work as intended
```

As we see from the code, the assignment to a primitive `5` is ignored.

We'll cover the special nature of `__proto__` in [subsequent chapters](#), and suggest the [ways to fix](#) such behavior.

Property existence test, “in” operator

A notable feature of objects in JavaScript, compared to many other languages, is that it's possible to access any property. There will be no error if the property doesn't exist!

Reading a non-existing property just returns `undefined`. So we can easily test whether the property exists:

```
let user = {};

alert( user.noSuchProperty === undefined ); // true means "no such property"
```

There's also a special operator `"in"` for that.

The syntax is:

```
"key" in object
```

For instance:

```
let user = { name: "John", age: 30 };

alert( "age" in user ); // true, user.age exists
alert( "blabla" in user ); // false, user.blabla doesn't exist
```

Please note that on the left side of `in` there must be a *property name*. That's usually a quoted string.

If we omit quotes, that means a variable should contain the actual name to be tested. For instance:

```
let user = { age: 30 };

let key = "age";
alert( key in user ); // true, property "age" exists
```

Why does the `in` operator exist? Isn't it enough to compare against `undefined`?

Well, most of the time the comparison with `undefined` works fine. But there's a special case when it fails, but `"in"` works correctly.

It's when an object property exists, but stores `undefined`:

```
let obj = {
  test: undefined
};
```

```
alert( obj.test ); // it's undefined, so - no such property?

alert( "test" in obj ); // true, the property does exist!
```

In the code above, the property `obj.test` technically exists. So the `in` operator works right.

Situations like this happen very rarely, because `undefined` should not be explicitly assigned. We mostly use `null` for “unknown” or “empty” values. So the `in` operator is an exotic guest in the code.

The "for..in" loop

To walk over all keys of an object, there exists a special form of the loop: `for..in`. This is a completely different thing from the `for(;;)` construct that we studied before.

The syntax:

```
for (key in object) {
  // executes the body for each key among object properties
}
```

For instance, let's output all properties of `user`:

```
let user = {
  name: "John",
  age: 30,
  isAdmin: true
};

for (let key in user) {
  // keys
  alert( key ); // name, age, isAdmin
  // values for the keys
  alert( user[key] ); // John, 30, true
}
```

Note that all “for” constructs allow us to declare the looping variable inside the loop, like `let key` here.

Also, we could use another variable name here instead of `key`. For instance, `"for (let prop in obj)"` is also widely used.

Ordered like an object

Are objects ordered? In other words, if we loop over an object, do we get all properties in the same order they were added? Can we rely on this?

The short answer is: “ordered in a special fashion”: integer properties are sorted, others appear in creation order. The details follow.

As an example, let's consider an object with the phone codes:

```
let codes = {
  "49": "Germany",
  "41": "Switzerland",
  "44": "Great Britain",
  // ..,
  "1": "USA"
};

for (let code in codes) {
  alert(code); // 1, 41, 44, 49
}
```

The object may be used to suggest a list of options to the user. If we're making a site mainly for a German audience then we probably want `49` to be the first.

But if we run the code, we see a totally different picture:

- USA (1) goes first

- then Switzerland (41) and so on.

The phone codes go in the ascending sorted order, because they are integers. So we see 1, 41, 44, 49.

Integer properties? What's that?

The “integer property” term here means a string that can be converted to-and-from an integer without a change.

So, “49” is an integer property name, because when it's transformed to an integer number and back, it's still the same. But “+49” and “1.2” are not:

```
// Number(...) explicitly converts to a number
// Math.trunc is a built-in function that removes the decimal part
alert( String(Math.trunc(Number("49"))) ); // "49", same, integer property
alert( String(Math.trunc(Number("+49"))) ); // "49", not same "+49" ⇒ not integer property
alert( String(Math.trunc(Number("1.2"))) ); // "1", not same "1.2" ⇒ not integer property
```

...On the other hand, if the keys are non-integer, then they are listed in the creation order, for instance:

```
let user = {
  name: "John",
  surname: "Smith"
};
user.age = 25; // add one more

// non-integer properties are listed in the creation order
for (let prop in user) {
  alert( prop ); // name, surname, age
}
```

So, to fix the issue with the phone codes, we can “cheat” by making the codes non-integer. Adding a plus “+” sign before each code is enough.

Like this:

```
let codes = {
  "+49": "Germany",
  "+41": "Switzerland",
  "+44": "Great Britain",
  // ..,
  "+1": "USA"
};

for (let code in codes) {
  alert( +code ); // 49, 41, 44, 1
}
```

Now it works as intended.

Summary

Objects are associative arrays with several special features.

They store properties (key-value pairs), where:

- Property keys must be strings or symbols (usually strings).
- Values can be of any type.

To access a property, we can use:

- The dot notation: `obj.property`.
- Square brackets notation `obj["property"]`. Square brackets allow taking the key from a variable, like `obj[varWithKey]`.

Additional operators:

- To delete a property: `delete obj.prop`.

- To check if a property with the given key exists: `"key" in obj`.
- To iterate over an object: `for (let key in obj)` loop.

What we've studied in this chapter is called a “plain object”, or just `Object`.

There are many other kinds of objects in JavaScript:

- `Array` to store ordered data collections,
- `Date` to store the information about the date and time,
- `Error` to store the information about an error.
- ...And so on.

They have their special features that we'll study later. Sometimes people say something like “Array type” or “Date type”, but formally they are not types of their own, but belong to a single “object” data type. And they extend it in various ways.

Objects in JavaScript are very powerful. Here we've just scratched the surface of a topic that is really huge. We'll be closely working with objects and learning more about them in further parts of the tutorial.

✓ Tasks

Hello, object

importance: 5

Write the code, one line for each action:

1. Create an empty object `user`.
2. Add the property `name` with the value `John`.
3. Add the property `surname` with the value `Smith`.
4. Change the value of the `name` to `Pete`.
5. Remove the property `name` from the object.

[To solution](#)

Check for emptiness

importance: 5

Write the function `isEmpty(obj)` which returns `true` if the object has no properties, `false` otherwise.

Should work like that:

```
let schedule = {};  
  
alert( isEmpty(schedule) ); // true  
  
schedule["8:30"] = "get up";  
  
alert( isEmpty(schedule) ); // false
```

[Open a sandbox with tests.](#) ↗

[To solution](#)

Sum object properties

importance: 5

We have an object storing salaries of our team:

```
let salaries = {  
  John: 100,  
  Ann: 160,  
  Pete: 130  
}
```

Write the code to sum all salaries and store in the variable `sum`. Should be `390` in the example above.

If `salaries` is empty, then the result must be `0`.

[To solution](#)

Multiply numeric property values by 2

importance: 3

Create a function `multiplyNumeric(obj)` that multiplies all numeric property values of `obj` by `2`.

For instance:

```
// before the call
let menu = {
  width: 200,
  height: 300,
  title: "My menu"
};

multiplyNumeric(menu);

// after the call
menu = {
  width: 400,
  height: 600,
  title: "My menu"
};
```

Please note that `multiplyNumeric` does not need to return anything. It should modify the object in-place.

P.S. Use `typeof` to check for a number here.

[Open a sandbox with tests.](#) ↗

[To solution](#)

Object references and copying

One of the fundamental differences of objects versus primitives is that objects are stored and copied “by reference”, whereas primitive values: strings, numbers, booleans, etc – are always copied “as a whole value”.

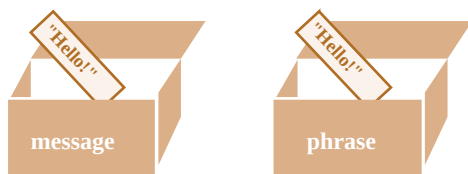
That’s easy to understand if we look a bit under the hood of what happens when we copy a value.

Let’s start with a primitive, such as a string.

Here we put a copy of `message` into `phrase`:

```
let message = "Hello!";
let phrase = message;
```

As a result we have two independent variables, each one storing the string `"Hello!"`.



Quite an obvious result, right?

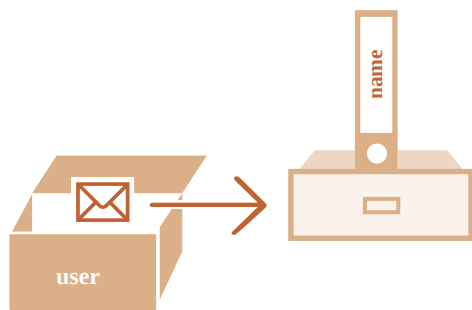
Objects are not like that.

A variable assigned to an object stores not the object itself, but its “address in memory” – in other words “a reference” to it.

Let's look at an example of such a variable:

```
let user = {  
  name: "John"  
};
```

And here's how it's actually stored in memory:



The object is stored somewhere in memory (at the right of the picture), while the `user` variable (at the left) has a “reference” to it.

We may think of an object variable, such as `user`, like a sheet of paper with the address of the object on it.

When we perform actions with the object, e.g. take a property `user.name`, the JavaScript engine looks at what's at that address and performs the operation on the actual object.

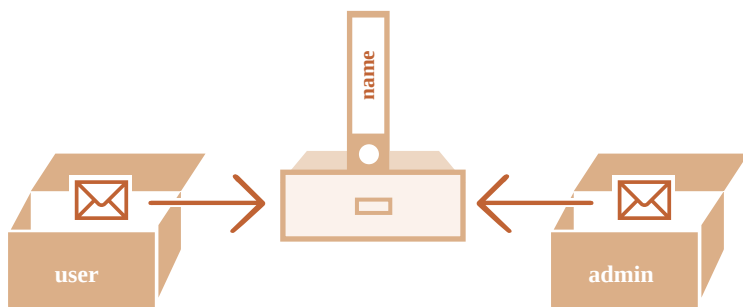
Now here's why it's important.

When an object variable is copied, the reference is copied, but the object itself is not duplicated.

For instance:

```
let user = { name: "John" };  
  
let admin = user; // copy the reference
```

Now we have two variables, each storing a reference to the same object:



As you can see, there's still one object, but now with two variables that reference it.

We can use either variable to access the object and modify its contents:

```
let user = { name: 'John' };  
  
let admin = user;  
  
admin.name = 'Pete'; // changed by the "admin" reference  
  
alert(user.name); // 'Pete', changes are seen from the "user" reference
```

It's as if we had a cabinet with two keys and used one of them (`admin`) to get into it and make changes. Then, if we later use another key (`user`), we are still opening the same cabinet and can access the changed contents.

Comparison by reference

Two objects are equal only if they are the same object.

For instance, here `a` and `b` reference the same object, thus they are equal:

```
let a = {};  
let b = a; // copy the reference  
  
alert( a == b ); // true, both variables reference the same object  
alert( a === b ); // true
```

And here two independent objects are not equal, even though they look alike (both are empty):

```
let a = {};  
let b = {}; // two independent objects  
  
alert( a == b ); // false
```

For comparisons like `obj1 > obj2` or for a comparison against a primitive `obj == 5`, objects are converted to primitives. We'll study how object conversions work very soon, but to tell the truth, such comparisons are needed very rarely – usually they appear as a result of a programming mistake.

Const objects can be modified

An important side effect of storing objects as references is that an object declared as `const` can be modified.

For instance:

```
const user = {  
  name: "John"  
};  
  
user.name = "Pete"; // (*)  
  
alert(user.name); // Pete
```

It might seem that the line `(*)` would cause an error, but it does not. The value of `user` is constant, it must always reference the same object, but properties of that object are free to change.

In other words, the `const user` gives an error only if we try to set `user=...` as a whole.

That said, if we really need to make constant object properties, it's also possible, but using totally different methods. We'll mention that in the chapter [Property flags and descriptors](#).

Cloning and merging, Object.assign

So, copying an object variable creates one more reference to the same object.

But what if we need to duplicate an object?

We can create a new object and replicate the structure of the existing one, by iterating over its properties and copying them on the primitive level.

Like this:

```
let user = {  
  name: "John",  
  age: 30  
};  
  
let clone = {}; // the new empty object  
  
// let's copy all user properties into it  
for (let key in user) {  
  clone[key] = user[key];  
}
```

```
// now clone is a fully independent object with the same content
clone.name = "Pete"; // changed the data in it

alert( user.name ); // still John in the original object
```

We can also use the method `Object.assign` [↗](#).

The syntax is:

```
Object.assign(dest, ...sources)
```

- The first argument `dest` is a target object.
- Further arguments is a list of source objects.

It copies the properties of all source objects into the target `dest`, and then returns it as the result.

For example, we have `user` object, let's add a couple of permissions to it:

```
let user = { name: "John" };

let permissions1 = { canView: true };
let permissions2 = { canEdit: true };

// copies all properties from permissions1 and permissions2 into user
Object.assign(user, permissions1, permissions2);

// now user = { name: "John", canView: true, canEdit: true }
alert(user.name); // John
alert(user.canView); // true
alert(user.canEdit); // true
```

If the copied property name already exists, it gets overwritten:

```
let user = { name: "John" };

Object.assign(user, { name: "Pete" });

alert(user.name); // now user = { name: "Pete" }
```

We also can use `Object.assign` to perform a simple object cloning:

```
let user = {
  name: "John",
  age: 30
};

let clone = Object.assign({}, user);

alert(clone.name); // John
alert(clone.age); // 30
```

Here it copies all properties of `user` into the empty object and returns it.

There are also other methods of cloning an object, e.g. using the [spread syntax](#) `clone = {...user}`, covered later in the tutorial.

Nested cloning

Until now we assumed that all properties of `user` are primitive. But properties can be references to other objects.

Like this:

```
let user = {
  name: "John",
```

```

    sizes: {
      height: 182,
      width: 50
    }
  };

  alert( user.sizes.height ); // 182

```

Now it's not enough to copy `clone.sizes = user.sizes`, because `user.sizes` is an object, and will be copied by reference, so `clone` and `user` will share the same sizes:

```

let user = {
  name: "John",
  sizes: {
    height: 182,
    width: 50
  }
};

let clone = Object.assign({}, user);

alert( user.sizes === clone.sizes ); // true, same object

// user and clone share sizes
user.sizes.width = 60;    // change a property from one place
alert(clone.sizes.width); // 60, get the result from the other one

```

To fix that and make `user` and `clone` truly separate objects, we should use a cloning loop that examines each value of `user[key]` and, if it's an object, then replicate its structure as well. That is called a “deep cloning” or “structured cloning”. There's [structuredClone](#)  method that implements deep cloning.

structuredClone

The call `structuredClone(object)` clones the `object` with all nested properties.

Here's how we can use it in our example:

```

let user = {
  name: "John",
  sizes: {
    height: 182,
    width: 50
  }
};

let clone = structuredClone(user);

alert( user.sizes === clone.sizes ); // false, different objects

// user and clone are totally unrelated now
user.sizes.width = 60;    // change a property from one place
alert(clone.sizes.width); // 50, not related

```

The `structuredClone` method can clone most data types, such as objects, arrays, primitive values.

It also supports circular references, when an object property references the object itself (directly or via a chain of references).

For instance:

```

let user = {};
// let's create a circular reference:
// user.me references the user itself
user.me = user;

let clone = structuredClone(user);
alert(clone.me === clone); // true

```

As you can see, `clone.me` references the `clone`, not the `user`! So the circular reference was cloned correctly as well.

Although, there are cases when `structuredClone` fails.

For instance, when an object has a function property:

```
// error
structuredClone({
  f: function() {}
});
```

Function properties aren't supported.

To handle such complex cases we may need to use a combination of cloning methods, write custom code or, to not reinvent the wheel, take an existing implementation, for instance `_.cloneDeep(obj)` [↗](#) from the JavaScript library `lodash` [↗](#).

Summary

Objects are assigned and copied by reference. In other words, a variable stores not the “object value”, but a “reference” (address in memory) for the value. So copying such a variable or passing it as a function argument copies that reference, not the object itself.

All operations via copied references (like adding/removing properties) are performed on the same single object.

To make a “real copy” (a clone) we can use `Object.assign` for the so-called “shallow copy” (nested objects are copied by reference) or a “deep cloning” function `structuredClone` or use a custom cloning implementation, such as `_.cloneDeep(obj)` [↗](#).

Garbage collection

Memory management in JavaScript is performed automatically and invisibly to us. We create primitives, objects, functions... All that takes memory.

What happens when something is not needed any more? How does the JavaScript engine discover it and clean it up?

Reachability

The main concept of memory management in JavaScript is *reachability*.

Simply put, “reachable” values are those that are accessible or usable somehow. They are guaranteed to be stored in memory.

1. There's a base set of inherently reachable values, that cannot be deleted for obvious reasons.

For instance:

- The currently executing function, its local variables and parameters.
- Other functions on the current chain of nested calls, their local variables and parameters.
- Global variables.
- (there are some other, internal ones as well)

These values are called *roots*.

2. Any other value is considered reachable if it's reachable from a root by a reference or by a chain of references.

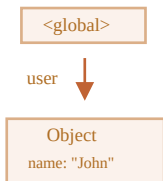
For instance, if there's an object in a global variable, and that object has a property referencing another object, *that* object is considered reachable. And those that it references are also reachable. Detailed examples to follow.

There's a background process in the JavaScript engine that is called [garbage collector](#) [↗](#). It monitors all objects and removes those that have become unreachable.

A simple example

Here's the simplest example:

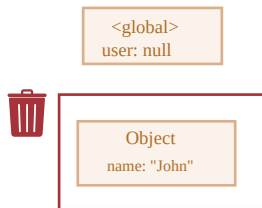
```
// user has a reference to the object
let user = {
  name: "John"
};
```



Here the arrow depicts an object reference. The global variable `"user"` references the object `{name: "John"}` (we'll call it John for brevity). The `"name"` property of John stores a primitive, so it's painted inside the object.

If the value of `user` is overwritten, the reference is lost:

```
user = null;
```



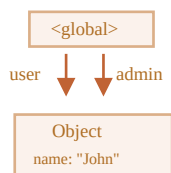
Now John becomes unreachable. There's no way to access it, no references to it. Garbage collector will junk the data and free the memory.

Two references

Now let's imagine we copied the reference from `user` to `admin`:

```
// user has a reference to the object
let user = {
  name: "John"
};

let admin = user;
```



Now if we do the same:

```
user = null;
```

...Then the object is still reachable via `admin` global variable, so it must stay in memory. If we overwrite `admin` too, then it can be removed.

Interlinked objects

Now a more complex example. The family:

```
function marry(man, woman) {
  woman.husband = man;
  man.wife = woman;

  return {
    father: man,
    mother: woman
  }
}
```

```

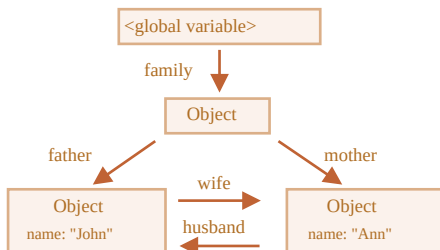
}

let family = marry({
  name: "John"
}, {
  name: "Ann"
});

```

Function `marry` “marries” two objects by giving them references to each other and returns a new object that contains them both.

The resulting memory structure:



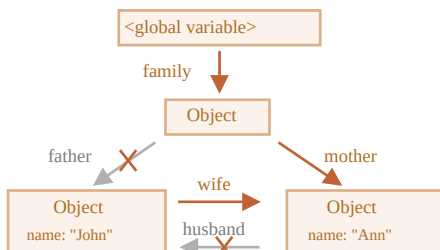
As of now, all objects are reachable.

Now let's remove two references:

```

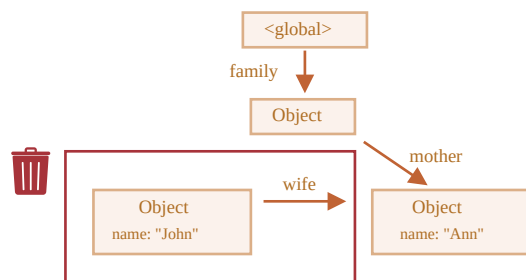
delete family.father;
delete family.mother.husband;

```



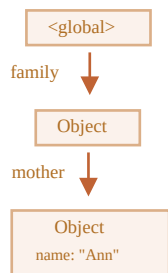
It's not enough to delete only one of these two references, because all objects would still be reachable.

But if we delete both, then we can see that John has no incoming reference any more:



Outgoing references do not matter. Only incoming ones can make an object reachable. So, John is now unreachable and will be removed from the memory with all its data that also became inaccessible.

After garbage collection:



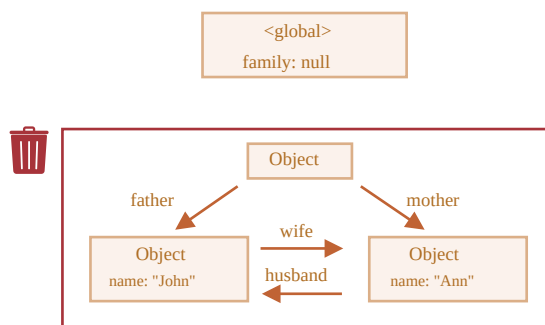
Unreachable island

It is possible that the whole island of interlinked objects becomes unreachable and is removed from the memory.

The source object is the same as above. Then:

```
family = null;
```

The in-memory picture becomes:



This example demonstrates how important the concept of reachability is.

It's obvious that John and Ann are still linked, both have incoming references. But that's not enough.

The former "family" object has been unlinked from the root, there's no reference to it any more, so the whole island becomes unreachable and will be removed.

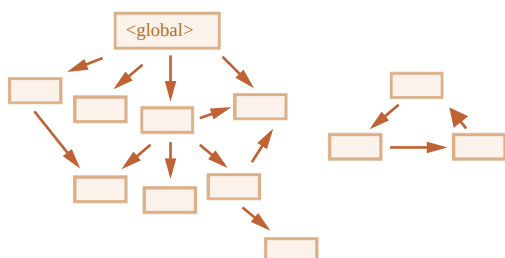
Internal algorithms

The basic garbage collection algorithm is called "mark-and-sweep".

The following "garbage collection" steps are regularly performed:

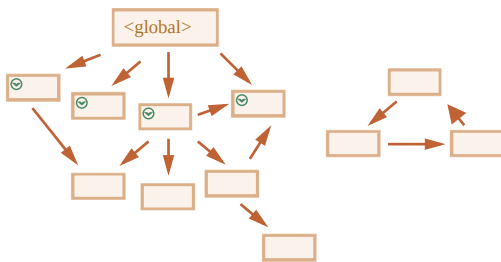
- The garbage collector takes roots and "marks" (remembers) them.
- Then it visits and "marks" all references from them.
- Then it visits marked objects and marks *their* references. All visited objects are remembered, so as not to visit the same object twice in the future.
- ...And so on until every reachable (from the roots) references are visited.
- All objects except marked ones are removed.

For instance, let our object structure look like this:

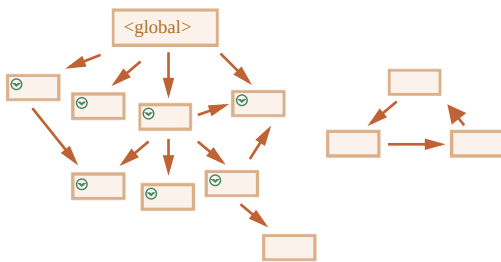


We can clearly see an “unreachable island” to the right side. Now let’s see how “mark-and-sweep” garbage collector deals with it.

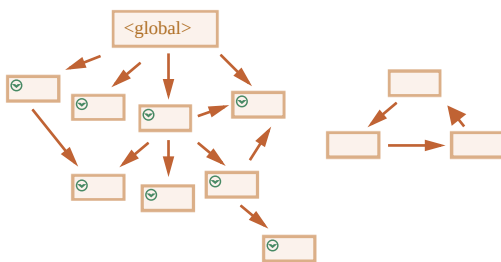
The first step marks the roots:



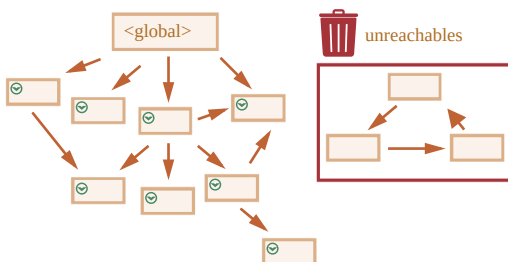
Then we follow their references and mark referenced objects:



...And continue to follow further references, while possible:



Now the objects that could not be visited in the process are considered unreachable and will be removed:



We can also imagine the process as spilling a huge bucket of paint from the roots, that flows through all references and marks all reachable objects. The unmarked ones are then removed.

That’s the concept of how garbage collection works. JavaScript engines apply many optimizations to make it run faster and not introduce any delays into the code execution.

Some of the optimizations:

- **Generational collection** – objects are split into two sets: “new ones” and “old ones”. In typical code, many objects have a short life span: they appear, do their job and die fast, so it makes sense to track new objects and clear the memory from them if that’s the case. Those that survive for long enough, become “old” and are examined less often.
- **Incremental collection** – if there are many objects, and we try to walk and mark the whole object set at once, it may take some time and introduce visible delays in the execution. So the engine splits the whole set of existing objects into multiple parts. And then clear these parts one after another. There are many small garbage collections instead of a total one. That requires some extra bookkeeping between them to track changes, but we get many tiny delays instead of a big one.
- **Idle-time collection** – the garbage collector tries to run only while the CPU is idle, to reduce the possible effect on the execution.

There exist other optimizations and flavours of garbage collection algorithms. As much as I'd like to describe them here, I have to hold off, because different engines implement different tweaks and techniques. And, what's even more important, things change as engines develop, so studying deeper "in advance", without a real need is probably not worth that. Unless, of course, it is a matter of pure interest, then there will be some links for you below.

Summary

The main things to know:

- Garbage collection is performed automatically. We cannot force or prevent it.
- Objects are retained in memory while they are reachable.
- Being referenced is not the same as being reachable (from a root): a pack of interlinked objects can become unreachable as a whole, as we've seen in the example above.

Modern engines implement advanced algorithms of garbage collection.

A general book "The Garbage Collection Handbook: The Art of Automatic Memory Management" (R. Jones et al) covers some of them.

If you are familiar with low-level programming, more detailed information about V8's garbage collector is in the article [A tour of V8: Garbage Collection](#) .

The [V8 blog](#) also publishes articles about changes in memory management from time to time. Naturally, to learn more about garbage collection, you'd better prepare by learning about V8 internals in general and read the blog of [Vyacheslav Egorov](#) who worked as one of the V8 engineers. I'm saying: "V8", because it is best covered by articles on the internet. For other engines, many approaches are similar, but garbage collection differs in many aspects.

In-depth knowledge of engines is good when you need low-level optimizations. It would be wise to plan that as the next step after you're familiar with the language.

Object methods, "this"

Objects are usually created to represent entities of the real world, like users, orders and so on:

```
let user = {  
  name: "John",  
  age: 30  
};
```

And, in the real world, a user can *act*: select something from the shopping cart, login, logout etc.

Actions are represented in JavaScript by functions in properties.

Method examples

For a start, let's teach the `user` to say hello:

```
let user = {  
  name: "John",  
  age: 30  
};  
  
user.sayHi = function() {  
  alert("Hello!");  
};  
  
user.sayHi(); // Hello!
```

Here we've just used a Function Expression to create a function and assign it to the property `user.sayHi` of the object.

Then we can call it as `user.sayHi()`. The user can now speak!

A function that is a property of an object is called its *method*.

So, here we've got a method `sayHi` of the object `user`.

Of course, we could use a pre-declared function as a method, like this:

```
let user = {
  // ...
};

// first, declare
function sayHi() {
  alert("Hello!");
}

// then add as a method
user.sayHi = sayHi;

user.sayHi(); // Hello!
```

Object-oriented programming

When we write our code using objects to represent entities, that's called [object-oriented programming](#) , in short: "OOP".

OOP is a big thing, an interesting science of its own. How to choose the right entities? How to organize the interaction between them? That's architecture, and there are great books on that topic, like "Design Patterns: Elements of Reusable Object-Oriented Software" by E. Gamma, R. Helm, R. Johnson, J. Vissides or "Object-Oriented Analysis and Design with Applications" by G. Booch, and more.

Method shorthand

There exists a shorter syntax for methods in an object literal:

```
// these objects do the same

user = {
  sayHi: function() {
    alert("Hello");
  }
};

// method shorthand looks better, right?
user = {
  sayHi() { // same as "sayHi: function(){...}"
    alert("Hello");
  }
};
```

As demonstrated, we can omit "function" and just write `sayHi()`.

To tell the truth, the notations are not fully identical. There are subtle differences related to object inheritance (to be covered later), but for now they do not matter. In almost all cases, the shorter syntax is preferred.

"this" in methods

It's common that an object method needs to access the information stored in the object to do its job.

For instance, the code inside `user.sayHi()` may need the name of the `user`.

To access the object, a method can use the `this` keyword.

The value of `this` is the object "before dot", the one used to call the method.

For instance:

```
let user = {
  name: "John",
  age: 30,

  sayHi() {
    // "this" is the "current object"
    alert(this.name);
  }
};
```

```
user.sayHi(); // John
```

Here during the execution of `user.sayHi()`, the value of `this` will be `user`.

Technically, it's also possible to access the object without `this`, by referencing it via the outer variable:

```
let user = {  
  name: "John",  
  age: 30,  
  
  sayHi() {  
    alert(user.name); // "user" instead of "this"  
  }  
  
};
```

...But such code is unreliable. If we decide to copy `user` to another variable, e.g. `admin = user` and overwrite `user` with something else, then it will access the wrong object.

That's demonstrated below:

```
let user = {  
  name: "John",  
  age: 30,  
  
  sayHi() {  
    alert( user.name ); // leads to an error  
  }  
  
};  
  
let admin = user;  
user = null; // overwrite to make things obvious  
  
admin.sayHi(); // TypeError: Cannot read property 'name' of null
```

If we used `this.name` instead of `user.name` inside the `alert`, then the code would work.

“this” is not bound

In JavaScript, keyword `this` behaves unlike most other programming languages. It can be used in any function, even if it's not a method of an object.

There's no syntax error in the following example:

```
function sayHi() {  
  alert( this.name );  
}
```

The value of `this` is evaluated during the run-time, depending on the context.

For instance, here the same function is assigned to two different objects and has different “this” in the calls:

```
let user = { name: "John" };  
let admin = { name: "Admin" };  
  
function sayHi() {  
  alert( this.name );  
}  
  
// use the same function in two objects  
user.f = sayHi;  
admin.f = sayHi;  
  
// these calls have different this  
// "this" inside the function is the object "before the dot"
```

```
user.f(); // John (this == user)
admin.f(); // Admin (this == admin)

admin['f'](); // Admin (dot or square brackets access the method – doesn't matter)
```

The rule is simple: if `obj.f()` is called, then `this` is `obj` during the call of `f`. So it's either `user` or `admin` in the example above.

i Calling without an object: `this == undefined`

We can even call the function without an object at all:

```
function sayHi() {
  alert(this);
}

sayHi(); // undefined
```

In this case `this` is `undefined` in strict mode. If we try to access `this.name`, there will be an error.

In non-strict mode the value of `this` in such case will be the *global object* (`window` in a browser, we'll get to it later in the chapter [Global object](#)). This is a historical behavior that `"use strict"` fixes.

Usually such call is a programming error. If there's `this` inside a function, it expects to be called in an object context.

i The consequences of unbound `this`

If you come from another programming language, then you are probably used to the idea of a “bound `this`”, where methods defined in an object always have `this` referencing that object.

In JavaScript `this` is “free”, its value is evaluated at call-time and does not depend on where the method was declared, but rather on what object is “before the dot”.

The concept of run-time evaluated `this` has both pluses and minuses. On the one hand, a function can be reused for different objects. On the other hand, the greater flexibility creates more possibilities for mistakes.

Here our position is not to judge whether this language design decision is good or bad. We'll understand how to work with it, how to get benefits and avoid problems.

Arrow functions have no “this”

Arrow functions are special: they don't have their “own” `this`. If we reference `this` from such a function, it's taken from the outer “normal” function.

For instance, here `arrow()` uses `this` from the outer `user.sayHi()` method:

```
let user = {
  firstName: "Ilya",
  sayHi() {
    let arrow = () => alert(this.firstName);
    arrow();
  }
};

user.sayHi(); // Ilya
```

That's a special feature of arrow functions, it's useful when we actually do not want to have a separate `this`, but rather to take it from the outer context. Later in the chapter [Arrow functions revisited](#) we'll go more deeply into arrow functions.

Summary

- Functions that are stored in object properties are called “methods”.
- Methods allow objects to “act” like `object.doSomething()`.
- Methods can reference the object as `this`.

The value of `this` is defined at run-time.

- When a function is declared, it may use `this`, but that `this` has no value until the function is called.
- A function can be copied between objects.
- When a function is called in the “method” syntax: `object.method()`, the value of `this` during the call is `object`.

Please note that arrow functions are special: they have no `this`. When `this` is accessed inside an arrow function, it is taken from outside.

✔ Tasks

Using "this" in object literal

importance: 5

Here the function `makeUser` returns an object.

What is the result of accessing its `ref`? Why?

```
function makeUser() {  
  return {  
    name: "John",  
    ref: this  
  };  
}  
  
let user = makeUser();  
  
alert( user.ref.name ); // What's the result?
```

[To solution](#)

Create a calculator

importance: 5

Create an object `calculator` with three methods:

- `read()` prompts for two values and saves them as object properties with names `a` and `b` respectively.
- `sum()` returns the sum of saved values.
- `mul()` multiplies saved values and returns the result.

```
let calculator = {  
  // ... your code ...  
};  
  
calculator.read();  
alert( calculator.sum() );  
alert( calculator.mul() );
```

[Run the demo](#)

[Open a sandbox with tests.](#) ↗

[To solution](#)

Chaining

importance: 2

There's a `ladder` object that allows you to go up and down:

```
let ladder = {  
  step: 0,  
  up() {  
    this.step++;  
  },  
  down() {
```

```

    this.step--;
  },
  showStep: function() { // shows the current step
    alert( this.step );
  }
};

```

Now, if we need to make several calls in sequence, we can do it like this:

```

ladder.up();
ladder.up();
ladder.down();
ladder.showStep(); // 1
ladder.down();
ladder.showStep(); // 0

```

Modify the code of `up`, `down`, and `showStep` to make the calls chainable, like this:

```

ladder.up().up().down().showStep().down().showStep(); // shows 1 then 0

```

Such an approach is widely used across JavaScript libraries.

[Open a sandbox with tests.](#) ↗

[To solution](#)

Constructor, operator "new"

The regular `{...}` syntax allows us to create one object. But often we need to create many similar objects, like multiple users or menu items and so on.

That can be done using constructor functions and the `"new"` operator.

Constructor function

Constructor functions technically are regular functions. There are two conventions though:

1. They are named with capital letter first.
2. They should be executed only with `"new"` operator.

For instance:

```

function User(name) {
  this.name = name;
  this.isAdmin = false;
}

let user = new User("Jack");

alert(user.name); // Jack
alert(user.isAdmin); // false

```

When a function is executed with `new`, it does the following steps:

1. A new empty object is created and assigned to `this`.
2. The function body executes. Usually it modifies `this`, adds new properties to it.
3. The value of `this` is returned.

In other words, `new User(...)` does something like:

```

function User(name) {
  // this = {}; (implicitly)

  // add properties to this

```

```

this.name = name;
this.isAdmin = false;

// return this; (implicitly)
}

```

So `let user = new User("Jack")` gives the same result as:

```

let user = {
  name: "Jack",
  isAdmin: false
};

```

Now if we want to create other users, we can call `new User("Ann")`, `new User("Alice")` and so on. Much shorter than using literals every time, and also easy to read.

That's the main purpose of constructors – to implement reusable object creation code.

Let's note once again – technically, any function (except arrow functions, as they don't have `this`) can be used as a constructor. It can be run with `new`, and it will execute the algorithm above. The “capital letter first” is a common agreement, to make it clear that a function is to be run with `new`.

new function() { ... }

If we have many lines of code all about creation of a single complex object, we can wrap them in an immediately called constructor function, like this:

```

// create a function and immediately call it with new
let user = new function() {
  this.name = "John";
  this.isAdmin = false;

  // ...other code for user creation
  // maybe complex logic and statements
  // local variables etc
};

```

This constructor can't be called again, because it is not saved anywhere, just created and called. So this trick aims to encapsulate the code that constructs the single object, without future reuse.

Constructor mode test: `new.target`

Advanced stuff

The syntax from this section is rarely used, skip it unless you want to know everything.

Inside a function, we can check whether it was called with `new` or without it, using a special `new.target` property.

It is undefined for regular calls and equals the function if called with `new`:

```

function User() {
  alert(new.target);
}

// without "new":
User(); // undefined

// with "new":
new User(); // function User { ... }

```

That can be used inside the function to know whether it was called with `new`, “in constructor mode”, or without it, “in regular mode”.

We can also make both `new` and regular calls to do the same, like this:

```
function User(name) {
  if (!new.target) { // if you run me without new
    return new User(name); // ...I will add new for you
  }

  this.name = name;
}

let john = User("John"); // redirects call to new User
alert(john.name); // John
```

This approach is sometimes used in libraries to make the syntax more flexible. So that people may call the function with or without `new`, and it still works.

Probably not a good thing to use everywhere though, because omitting `new` makes it a bit less obvious what's going on. With `new` we all know that the new object is being created.

Return from constructors

Usually, constructors do not have a `return` statement. Their task is to write all necessary stuff into `this`, and it automatically becomes the result.

But if there is a `return` statement, then the rule is simple:

- If `return` is called with an object, then the object is returned instead of `this`.
- If `return` is called with a primitive, it's ignored.

In other words, `return` with an object returns that object, in all other cases `this` is returned.

For instance, here `return` overrides `this` by returning an object:

```
function BigUser() {
  this.name = "John";

  return { name: "Godzilla" }; // <-- returns this object
}

alert( new BigUser().name ); // Godzilla, got that object
```

And here's an example with an empty `return` (or we could place a primitive after it, doesn't matter):

```
function SmallUser() {
  this.name = "John";

  return; // <-- returns this
}

alert( new SmallUser().name ); // John
```

Usually constructors don't have a `return` statement. Here we mention the special behavior with returning objects mainly for the sake of completeness.

Omitting parentheses

By the way, we can omit parentheses after `new`:

```
let user = new User; // <-- no parentheses
// same as
let user = new User();
```

Omitting parentheses here is not considered a “good style”, but the syntax is permitted by specification.

Methods in constructor

Using constructor functions to create objects gives a great deal of flexibility. The constructor function may have parameters that define how to construct the object, and what to put in it.

Of course, we can add to `this` not only properties, but methods as well.

For instance, `new User(name)` below creates an object with the given `name` and the method `sayHi`:

```
function User(name) {
  this.name = name;

  this.sayHi = function() {
    alert( "My name is: " + this.name );
  };
}

let john = new User("John");

john.sayHi(); // My name is: John

/*
john = {
  name: "John",
  sayHi: function() { ... }
}
*/
```

To create complex objects, there's a more advanced syntax, [classes](#), that we'll cover later.

Summary

- Constructor functions or, briefly, constructors, are regular functions, but there's a common agreement to name them with capital letter first.
- Constructor functions should only be called using `new`. Such a call implies a creation of empty `this` at the start and returning the populated one at the end.

We can use constructor functions to make multiple similar objects.

JavaScript provides constructor functions for many built-in language objects: like `Date` for dates, `Set` for sets and others that we plan to study.

Objects, we'll be back!

In this chapter we only cover the basics about objects and constructors. They are essential for learning more about data types and functions in the next chapters.

After we learn that, we return to objects and cover them in-depth in the chapters [Prototypes](#), [inheritance](#) and [Classes](#).

Tasks

Two functions – one object

importance: 2

Is it possible to create functions `A` and `B` so that `new A() == new B()`?

```
function A() { ... }
function B() { ... }

let a = new A();
let b = new B();

alert( a == b ); // true
```

If it is, then provide an example of their code.

[To solution](#)

Create new Calculator

importance: 5

Create a constructor function `Calculator` that creates objects with 3 methods:

- `read()` prompts for two values and saves them as object properties with names `a` and `b` respectively.
- `sum()` returns the sum of these properties.
- `mul()` returns the multiplication product of these properties.

For instance:

```
let calculator = new Calculator();
calculator.read();

alert( "Sum=" + calculator.sum() );
alert( "Mul=" + calculator.mul() );
```

[Run the demo](#)

[Open a sandbox with tests.](#) ↗

[To solution](#)

Create new Accumulator

importance: 5

Create a constructor function `Accumulator(startingValue)`.

Object that it creates should:

- Store the “current value” in the property `value`. The starting value is set to the argument of the constructor `startingValue`.
- The `read()` method should use `prompt` to read a new number and add it to `value`.

In other words, the `value` property is the sum of all user-entered values with the initial value `startingValue`.

Here's the demo of the code:

```
let accumulator = new Accumulator(1); // initial value 1

accumulator.read(); // adds the user-entered value
accumulator.read(); // adds the user-entered value

alert(accumulator.value); // shows the sum of these values
```

[Run the demo](#)

[Open a sandbox with tests.](#) ↗

[To solution](#)

Optional chaining '?.'

A recent addition

This is a recent addition to the language. Old browsers may need [polyfills](#).

The optional chaining `?.` is a safe way to access nested object properties, even if an intermediate property doesn't exist.

The “non-existing property” problem

If you’ve just started to read the tutorial and learn JavaScript, maybe the problem hasn’t touched you yet, but it’s quite common.

As an example, let’s say we have `user` objects that hold the information about our users.

Most of our users have addresses in `user.address` property, with the street `user.address.street`, but some did not provide them.

In such case, when we attempt to get `user.address.street`, and the user happens to be without an address, we get an error:

```
let user = {}; // a user without "address" property
alert(user.address.street); // Error!
```

That’s the expected result. JavaScript works like this. As `user.address` is `undefined`, an attempt to get `user.address.street` fails with an error.

In many practical cases we’d prefer to get `undefined` instead of an error here (meaning “no street”).

...and another example. In Web development, we can get an object that corresponds to a web page element using a special method call, such as `document.querySelector('.elem')`, and it returns `null` when there’s no such element.

```
// document.querySelector('.elem') is null if there's no element
let html = document.querySelector('.elem').innerHTML; // error if it's null
```

Once again, if the element doesn’t exist, we’ll get an error accessing `.innerHTML` property of `null`. And in some cases, when the absence of the element is normal, we’d like to avoid the error and just accept `html = null` as the result.

How can we do this?

The obvious solution would be to check the value using `if` or the conditional operator `?`, before accessing its property, like this:

```
let user = {};
alert(user.address ? user.address.street : undefined);
```

It works, there’s no error... But it’s quite inelegant. As you can see, the `"user.address"` appears twice in the code.

Here’s how the same would look for `document.querySelector`:

```
let html = document.querySelector('.elem') ? document.querySelector('.elem').innerHTML : null;
```

We can see that the element search `document.querySelector('.elem')` is actually called twice here. Not good.

For more deeply nested properties, it becomes even uglier, as more repetitions are required.

E.g. let’s get `user.address.street.name` in a similar fashion.

```
let user = {}; // user has no address
alert(user.address ? user.address.street ? user.address.street.name : null : null);
```

That’s just awful, one may even have problems understanding such code.

There’s a little better way to write it, using the `&&` operator:

```
let user = {}; // user has no address
alert( user.address && user.address.street && user.address.street.name ); // undefined (no error)
```

AND’ing the whole path to the property ensures that all components exist (if not, the evaluation stops), but also isn’t ideal.

As you can see, property names are still duplicated in the code. E.g. in the code above, `user.address` appears three times.

That's why the optional chaining `?.` was added to the language. To solve this problem once and for all!

Optional chaining

The optional chaining `?.` stops the evaluation if the value before `?.` is `undefined` or `null` and returns `undefined`.

Further in this article, for brevity, we'll be saying that something "exists" if it's not `null` and not `undefined`.

In other words, `value?.prop`:

- works as `value.prop`, if `value` exists,
- otherwise (when `value` is `undefined/null`) it returns `undefined`.

Here's the safe way to access `user.address.street` using `?.`:

```
let user = {}; // user has no address

alert( user?.address?.street ); // undefined (no error)
```

The code is short and clean, there's no duplication at all.

Here's an example with `document.querySelector`:

```
let html = document.querySelector('.elem')?.innerHTML; // will be undefined, if there's no element
```

Reading the address with `user?.address` works even if `user` object doesn't exist:

```
let user = null;

alert( user?.address ); // undefined
alert( user?.address.street ); // undefined
```

Please note: the `?.` syntax makes optional the value before it, but not any further.

E.g. in `user?.address.street.name` the `?.` allows `user` to safely be `null/undefined` (and returns `undefined` in that case), but that's only for `user`. Further properties are accessed in a regular way. If we want some of them to be optional, then we'll need to replace more `.` with `?.`.

⚠ Don't overuse the optional chaining

We should use `?.` only where it's ok that something doesn't exist.

For example, if according to our code logic `user` object must exist, but `address` is optional, then we should write `user.address?.street`, but not `user?.address?.street`.

Then, if `user` happens to be `undefined`, we'll see a programming error about it and fix it. Otherwise, if we overuse `?.`, coding errors can be silenced where not appropriate, and become more difficult to debug.

⚠ The variable before `?.` must be declared

If there's no variable `user` at all, then `user?.anything` triggers an error:

```
// ReferenceError: user is not defined
user?.address;
```

The variable must be declared (e.g. `let/const/var user` or as a function parameter). The optional chaining works only for declared variables.

Short-circuiting

As it was said before, the `?.` immediately stops ("short-circuits") the evaluation if the left part doesn't exist.

So, if there are any further function calls or operations to the right of `?.`, they won't be made.

For instance:

```
let user = null;
let x = 0;

user?.sayHi(x++); // no "user", so the execution doesn't reach sayHi call and x++

alert(x); // 0, value not incremented
```

Other variants: `?.()`, `?.[]`

The optional chaining `?.` is not an operator, but a special syntax construct, that also works with functions and square brackets.

For example, `?.()` is used to call a function that may not exist.

In the code below, some of our users have `admin` method, and some don't:

```
let userAdmin = {
  admin() {
    alert("I am admin");
  }
};

let userGuest = {};

userAdmin.admin?.(); // I am admin

userGuest.admin?.(); // nothing happens (no such method)
```

Here, in both lines we first use the dot (`userAdmin.admin`) to get `admin` property, because we assume that the `user` object exists, so it's safe read from it.

Then `?.()` checks the left part: if the `admin` function exists, then it runs (that's so for `userAdmin`). Otherwise (for `userGuest`) the evaluation stops without errors.

The `?.[]` syntax also works, if we'd like to use brackets `[]` to access properties instead of dot `.`. Similar to previous cases, it allows to safely read a property from an object that may not exist.

```
let key = "firstName";

let user1 = {
  firstName: "John"
};

let user2 = null;

alert( user1?.[key] ); // John
alert( user2?.[key] ); // undefined
```

Also we can use `?.` with `delete`:

```
delete user?.name; // delete user.name if user exists
```

We can use `?.` for safe reading and deleting, but not writing

The optional chaining `?.` has no use on the left side of an assignment.

For example:

```
let user = null;

user?.name = "John"; // Error, doesn't work
// because it evaluates to: undefined = "John"
```

Summary

The optional chaining `?.` syntax has three forms:

1. `obj?.prop` – returns `obj.prop` if `obj` exists, otherwise `undefined`.
2. `obj?.[prop]` – returns `obj[prop]` if `obj` exists, otherwise `undefined`.
3. `obj.method?.()` – calls `obj.method()` if `obj.method` exists, otherwise returns `undefined`.

As we can see, all of them are straightforward and simple to use. The `?.` checks the left part for `null/undefined` and allows the evaluation to proceed if it's not so.

A chain of `?.` allows to safely access nested properties.

Still, we should apply `?.` carefully, only where it's acceptable, according to our code logic, that the left part doesn't exist. So that it won't hide programming errors from us, if they occur.

Symbol type

By specification, only two primitive types may serve as object property keys:

- string type, or
- symbol type.

Otherwise, if one uses another type, such as number, it's autoconverted to string. So that `obj[1]` is the same as `obj["1"]`, and `obj[true]` is the same as `obj["true"]`.

Until now we've been using only strings.

Now let's explore symbols, see what they can do for us.

Symbols

A “symbol” represents a unique identifier.

A value of this type can be created using `Symbol()`:

```
let id = Symbol();
```

Upon creation, we can give symbols a description (also called a symbol name), mostly useful for debugging purposes:

```
// id is a symbol with the description "id"
let id = Symbol("id");
```

Symbols are guaranteed to be unique. Even if we create many symbols with exactly the same description, they are different values. The description is just a label that doesn't affect anything.

For instance, here are two symbols with the same description – they are not equal:

```
let id1 = Symbol("id");
let id2 = Symbol("id");

alert(id1 == id2); // false
```

If you are familiar with Ruby or another language that also has some sort of “symbols” – please don’t be misguided. JavaScript symbols are different.

So, to summarize, a symbol is a “primitive unique value” with an optional description. Let’s see where we can use them.

⚠ Symbols don’t auto-convert to a string

Most values in JavaScript support implicit conversion to a string. For instance, we can `alert` almost any value, and it will work. Symbols are special. They don’t auto-convert.

For instance, this `alert` will show an error:

```
let id = Symbol("id");
alert(id); // TypeError: Cannot convert a Symbol value to a string
```

That’s a “language guard” against messing up, because strings and symbols are fundamentally different and should not accidentally convert one into another.

If we really want to show a symbol, we need to explicitly call `.toString()` on it, like here:

```
let id = Symbol("id");
alert(id.toString()); // Symbol(id), now it works
```

Or get `symbol.description` property to show the description only:

```
let id = Symbol("id");
alert(id.description); // id
```

“Hidden” properties

Symbols allow us to create “hidden” properties of an object, that no other part of code can accidentally access or overwrite.

For instance, if we’re working with `user` objects, that belong to a third-party code. We’d like to add identifiers to them.

Let’s use a symbol key for it:

```
let user = { // belongs to another code
  name: "John"
};

let id = Symbol("id");

user[id] = 1;

alert( user[id] ); // we can access the data using the symbol as the key
```

What’s the benefit of using `Symbol("id")` over a string `"id"`?

As `user` objects belong to another codebase, it’s unsafe to add fields to them, since we might affect pre-defined behavior in that other codebase. However, symbols cannot be accessed accidentally. The third-party code won’t be aware of newly defined symbols, so it’s safe to add symbols to the `user` objects.

Also, imagine that another script wants to have its own identifier inside `user`, for its own purposes.

Then that script can create its own `Symbol("id")`, like this:

```
// ...
let id = Symbol("id");

user[id] = "Their id value";
```

There will be no conflict between our and their identifiers, because symbols are always different, even if they have the same name.

...But if we used a string `"id"` instead of a symbol for the same purpose, then there *would* be a conflict:

```
let user = { name: "John" };

// Our script uses "id" property
user.id = "Our id value";

// ...Another script also wants "id" for its purposes...

user.id = "Their id value"
// Boom! overwritten by another script!
```

Symbols in an object literal

If we want to use a symbol in an object literal `{...}`, we need square brackets around it.

Like this:

```
let id = Symbol("id");

let user = {
  name: "John",
  [id]: 123 // not "id": 123
};
```

That's because we need the value from the variable `id` as the key, not the string `"id"`.

Symbols are skipped by `for...in`

Symbolic properties do not participate in `for...in` loop.

For instance:

```
let id = Symbol("id");
let user = {
  name: "John",
  age: 30,
  [id]: 123
};

for (let key in user) alert(key); // name, age (no symbols)

// the direct access by the symbol works
alert( "Direct: " + user[id] ); // Direct: 123
```

`Object.keys(user)` [↗](#) also ignores them. That's a part of the general “hiding symbolic properties” principle. If another script or a library loops over our object, it won't unexpectedly access a symbolic property.

In contrast, `Object.assign` [↗](#) copies both string and symbol properties:

```
let id = Symbol("id");
let user = {
  [id]: 123
};

let clone = Object.assign({}, user);

alert( clone[id] ); // 123
```

There's no paradox here. That's by design. The idea is that when we clone an object or merge objects, we usually want *all* properties to be copied (including symbols like `id`).

Global symbols

As we've seen, usually all symbols are different, even if they have the same name. But sometimes we want same-named symbols to be same entities. For instance, different parts of our application want to access symbol `"id"` meaning exactly the same property.

To achieve that, there exists a *global symbol registry*. We can create symbols in it and access them later, and it guarantees that repeated accesses by the same name return exactly the same symbol.

In order to read (create if absent) a symbol from the registry, use `Symbol.for(key)`.

That call checks the global registry, and if there's a symbol described as `key`, then returns it, otherwise creates a new symbol `Symbol(key)` and stores it in the registry by the given `key`.

For instance:

```
// read from the global registry
let id = Symbol.for("id"); // if the symbol did not exist, it is created

// read it again (maybe from another part of the code)
let idAgain = Symbol.for("id");

// the same symbol
alert( id === idAgain ); // true
```

Symbols inside the registry are called *global symbols*. If we want an application-wide symbol, accessible everywhere in the code – that's what they are for.

That sounds like Ruby

In some programming languages, like Ruby, there's a single symbol per name.

In JavaScript, as we can see, that's true for global symbols.

Symbol.keyFor

We have seen that for global symbols, `Symbol.for(key)` returns a symbol by name. To do the opposite – return a name by global symbol – we can use: `Symbol.keyFor(sym)`:

For instance:

```
// get symbol by name
let sym = Symbol.for("name");
let sym2 = Symbol.for("id");

// get name by symbol
alert( Symbol.keyFor(sym) ); // name
alert( Symbol.keyFor(sym2) ); // id
```

The `Symbol.keyFor` internally uses the global symbol registry to look up the key for the symbol. So it doesn't work for non-global symbols. If the symbol is not global, it won't be able to find it and returns `undefined`.

That said, all symbols have the `description` property.

For instance:

```
let globalSymbol = Symbol.for("name");
let localSymbol = Symbol("name");

alert( Symbol.keyFor(globalSymbol) ); // name, global symbol
alert( Symbol.keyFor(localSymbol) ); // undefined, not global

alert( localSymbol.description ); // name
```

System symbols

There exist many “system” symbols that JavaScript uses internally, and we can use them to fine-tune various aspects of our objects.

They are listed in the specification in the [Well-known symbols](#)  table:

- `Symbol.hasInstance`
- `Symbol.isConcatSpreadable`
- `Symbol.iterator`
- `Symbol.toPrimitive`
- ...and so on.

For instance, `Symbol.toPrimitive` allows us to describe object to primitive conversion. We'll see its use very soon. Other symbols will also become familiar when we study the corresponding language features.

Summary

`Symbol` is a primitive type for unique identifiers.

Symbols are created with `Symbol()` call with an optional description (name).

Symbols are always different values, even if they have the same name. If we want same-named symbols to be equal, then we should use the global registry: `Symbol.for(key)` returns (creates if needed) a global symbol with `key` as the name. Multiple calls of `Symbol.for` with the same `key` return exactly the same symbol.

Symbols have two main use cases:

1. "Hidden" object properties.

If we want to add a property into an object that "belongs" to another script or a library, we can create a symbol and use it as a property key. A symbolic property does not appear in `for...in`, so it won't be accidentally processed together with other properties. Also it won't be accessed directly, because another script does not have our symbol. So the property will be protected from accidental use or overwrite.

So we can "covertly" hide something into objects that we need, but others should not see, using symbolic properties.

2. There are many system symbols used by JavaScript which are accessible as `Symbol.*`. We can use them to alter some built-in behaviors. For instance, later in the tutorial we'll use `Symbol.iterator` for `iterables`, `Symbol.toPrimitive` to setup `object-to-primitive conversion` and so on.

Technically, symbols are not 100% hidden. There is a built-in method `Object.getOwnPropertySymbols(obj)` that allows us to get all symbols. Also there is a method named `Reflect.ownKeys(obj)` that returns *all* keys of an object including symbolic ones. But most libraries, built-in functions and syntax constructs don't use these methods.

Object to primitive conversion

What happens when objects are added `obj1 + obj2`, subtracted `obj1 - obj2` or printed using `alert(obj)`?

JavaScript doesn't allow you to customize how operators work on objects. Unlike some other programming languages, such as Ruby or C++, we can't implement a special object method to handle addition (or other operators).

In case of such operations, objects are auto-converted to primitives, and then the operation is carried out over these primitives and results in a primitive value.

That's an important limitation: the result of `obj1 + obj2` (or another math operation) can't be another object!

E.g. we can't make objects representing vectors or matrices (or achievements or whatever), add them and expect a "summed" object as the result. Such architectural feats are automatically "off the board".

So, because we can't technically do much here, there's no maths with objects in real projects. When it happens, with rare exceptions, it's because of a coding mistake.

In this chapter we'll cover how an object converts to primitive and how to customize it.

We have two purposes:

1. It will allow us to understand what's going on in case of coding mistakes, when such an operation happened accidentally.
2. There are exceptions, where such operations are possible and look good. E.g. subtracting or comparing dates (`Date` objects). We'll come across them later.

Conversion rules

In the chapter [Type Conversions](#) we've seen the rules for numeric, string and boolean conversions of primitives. But we left a gap for objects. Now, as we know about methods and symbols it becomes possible to fill it.

1. There's no conversion to boolean. All objects are `true` in a boolean context, as simple as that. There exist only numeric and string conversions.
2. The numeric conversion happens when we subtract objects or apply mathematical functions. For instance, `Date` objects (to be covered in the chapter [Date and time](#)) can be subtracted, and the result of `date1 - date2` is the time difference between two dates.
3. As for the string conversion – it usually happens when we output an object with `alert(obj)` and in similar contexts.

We can implement string and numeric conversion by ourselves, using special object methods.

Now let's get into technical details, because it's the only way to cover the topic in-depth.

Hints

How does JavaScript decide which conversion to apply?

There are three variants of type conversion, that happen in various situations. They're called "hints", as described in the [specification](#) ↗:

"string"

For an object-to-string conversion, when we're doing an operation on an object that expects a string, like `alert`:

```
// output
alert(obj);

// using object as a property key
anotherObj[obj] = 123;
```

"number"

For an object-to-number conversion, like when we're doing maths:

```
// explicit conversion
let num = Number(obj);

// maths (except binary plus)
let n = +obj; // unary plus
let delta = date1 - date2;

// less/greater comparison
let greater = user1 > user2;
```

Most built-in mathematical functions also include such conversion.

"default"

Occurs in rare cases when the operator is "not sure" what type to expect.

For instance, binary plus `+` can work both with strings (concatenates them) and numbers (adds them). So if a binary plus gets an object as an argument, it uses the "default" hint to convert it.

Also, if an object is compared using `==` with a string, number or a symbol, it's also unclear which conversion should be done, so the "default" hint is used.

```
// binary plus uses the "default" hint
let total = obj1 + obj2;

// obj == number uses the "default" hint
if (user == 1) { ... };
```

The greater and less comparison operators, such as `<` `>`, can work with both strings and numbers too. Still, they use the "number" hint, not "default". That's for historical reasons.

In practice though, things are a bit simpler.

All built-in objects except for one case (`Date` object, we'll learn it later) implement "default" conversion the same way as "number". And we probably should do the same.

Still, it's important to know about all 3 hints, soon we'll see why.

To do the conversion, JavaScript tries to find and call three object methods:

1. Call `obj[Symbol.toPrimitive](hint)` – the method with the symbolic key `Symbol.toPrimitive` (system symbol), if such method exists,
2. Otherwise if hint is "string"
 - try calling `obj.toString()` or `obj.valueOf()`, whatever exists.

3. Otherwise if hint is "number" or "default"
- try calling `obj.valueOf()` or `obj.toString()`, whatever exists.

Symbol.toPrimitive

Let's start from the first method. There's a built-in symbol named `Symbol.toPrimitive` that should be used to name the conversion method, like this:

```
obj[Symbol.toPrimitive] = function(hint) {  
  // here goes the code to convert this object to a primitive  
  // it must return a primitive value  
  // hint = one of "string", "number", "default"  
};
```

If the method `Symbol.toPrimitive` exists, it's used for all hints, and no more methods are needed.

For instance, here `user` object implements it:

```
let user = {  
  name: "John",  
  money: 1000,  
  
  [Symbol.toPrimitive](hint) {  
    alert(`hint: ${hint}`);  
    return hint == "string" ? `{name: "${this.name}"}` : this.money;  
  }  
};  
  
// conversions demo:  
alert(user); // hint: string -> {name: "John"}  
alert(+user); // hint: number -> 1000  
alert(user + 500); // hint: default -> 1500
```

As we can see from the code, `user` becomes a self-descriptive string or a money amount, depending on the conversion. The single method `user[Symbol.toPrimitive]` handles all conversion cases.

toString/valueOf

If there's no `Symbol.toPrimitive` then JavaScript tries to find methods `toString` and `valueOf`:

- For the "string" hint: call `toString` method, and if it doesn't exist or if it returns an object instead of a primitive value, then call `valueOf` (so `toString` has the priority for string conversions).
- For other hints: call `valueOf`, and if it doesn't exist or if it returns an object instead of a primitive value, then call `toString` (so `valueOf` has the priority for maths).

Methods `toString` and `valueOf` come from ancient times. They are not symbols (symbols did not exist that long ago), but rather "regular" string-named methods. They provide an alternative "old-style" way to implement the conversion.

These methods must return a primitive value. If `toString` or `valueOf` returns an object, then it's ignored (same as if there were no method).

By default, a plain object has following `toString` and `valueOf` methods:

- The `toString` method returns a string `"[object Object]"`.
- The `valueOf` method returns the object itself.

Here's the demo:

```
let user = {name: "John"};  
  
alert(user); // [object Object]  
alert(user.valueOf() === user); // true
```

So if we try to use an object as a string, like in an `alert` or so, then by default we see `[object Object]`.

The default `valueOf` is mentioned here only for the sake of completeness, to avoid any confusion. As you can see, it returns the object itself, and so is ignored. Don't ask me why, that's for historical reasons. So we can assume it doesn't exist.

Let's implement these methods to customize the conversion.

For instance, here `user` does the same as above using a combination of `toString` and `valueOf` instead of `Symbol.toPrimitive`:

```
let user = {
  name: "John",
  money: 1000,

  // for hint="string"
  toString() {
    return `{name: "${this.name}"}`;
  },

  // for hint="number" or "default"
  valueOf() {
    return this.money;
  }
};

alert(user); // toString -> {name: "John"}
alert(+user); // valueOf -> 1000
alert(user + 500); // valueOf -> 1500
```

As we can see, the behavior is the same as the previous example with `Symbol.toPrimitive`.

Often we want a single “catch-all” place to handle all primitive conversions. In this case, we can implement `toString` only, like this:

```
let user = {
  name: "John",

  toString() {
    return this.name;
  }
};

alert(user); // toString -> John
alert(user + 500); // toString -> John500
```

In the absence of `Symbol.toPrimitive` and `valueOf`, `toString` will handle all primitive conversions.

A conversion can return any primitive type

The important thing to know about all primitive-conversion methods is that they do not necessarily return the “hinted” primitive.

There is no control whether `toString` returns exactly a string, or whether `Symbol.toPrimitive` method returns a number for the hint `"number"`.

The only mandatory thing: these methods must return a primitive, not an object.

Historical notes

For historical reasons, if `toString` or `valueOf` returns an object, there's no error, but such value is ignored (like if the method didn't exist). That's because in ancient times there was no good “error” concept in JavaScript.

In contrast, `Symbol.toPrimitive` is stricter, it *must* return a primitive, otherwise there will be an error.

Further conversions

As we know already, many operators and functions perform type conversions, e.g. multiplication `*` converts operands to numbers.

If we pass an object as an argument, then there are two stages of calculations:

1. The object is converted to a primitive (using the rules described above).

2. If necessary for further calculations, the resulting primitive is also converted.

For instance:

```
let obj = {
  // toString handles all conversions in the absence of other methods
  toString() {
    return "2";
  }
};

alert(obj * 2); // 4, object converted to primitive "2", then multiplication made it a number
```

1. The multiplication `obj * 2` first converts the object to primitive (that's a string `"2"`).
2. Then `"2" * 2` becomes `2 * 2` (the string is converted to number).

Binary plus will concatenate strings in the same situation, as it gladly accepts a string:

```
let obj = {
  toString() {
    return "2";
  }
};

alert(obj + 2); // "22" ("2" + 2), conversion to primitive returned a string => concatenation
```

Summary

The object-to-primitive conversion is called automatically by many built-in functions and operators that expect a primitive as a value.

There are 3 types (hints) of it:

- `"string"` (for `alert` and other operations that need a string)
- `"number"` (for maths)
- `"default"` (few operators, usually objects implement it the same way as `"number"`)

The specification describes explicitly which operator uses which hint.

The conversion algorithm is:

1. Call `obj[Symbol.toPrimitive](hint)` if the method exists,
2. Otherwise if hint is `"string"`
 - try calling `obj.toString()` or `obj.valueOf()`, whatever exists.
3. Otherwise if hint is `"number"` or `"default"`
 - try calling `obj.valueOf()` or `obj.toString()`, whatever exists.

All these methods must return a primitive to work (if defined).

In practice, it's often enough to implement only `obj.toString()` as a "catch-all" method for string conversions that should return a "human-readable" representation of an object, for logging or debugging purposes.

Data types

More data structures and more in-depth study of the types.

Methods of primitives

JavaScript allows us to work with primitives (strings, numbers, etc.) as if they were objects. They also provide methods to call as such. We will study those soon, but first we'll see how it works because, of course, primitives are not objects (and here we will make it even clearer).

Let's look at the key distinctions between primitives and objects.

A primitive

- Is a value of a primitive type.
- There are 7 primitive types: `string`, `number`, `bigint`, `boolean`, `symbol`, `null` and `undefined`.

An object

- Is capable of storing multiple values as properties.
- Can be created with `{}`, for instance: `{name: "John", age: 30}`. There are other kinds of objects in JavaScript: functions, for example, are objects.

One of the best things about objects is that we can store a function as one of its properties.

```
let john = {
  name: "John",
  sayHi: function() {
    alert("Hi buddy!");
  }
};

john.sayHi(); // Hi buddy!
```

So here we've made an object `john` with the method `sayHi`.

Many built-in objects already exist, such as those that work with dates, errors, HTML elements, etc. They have different properties and methods.

But, these features come with a cost!

Objects are “heavier” than primitives. They require additional resources to support the internal machinery.

A primitive as an object

Here's the paradox faced by the creator of JavaScript:

- There are many things one would want to do with a primitive, like a string or a number. It would be great to access them using methods.
- Primitives must be as fast and lightweight as possible.

The solution looks a little bit awkward, but here it is:

1. Primitives are still primitive. A single value, as desired.
2. The language allows access to methods and properties of strings, numbers, booleans and symbols.
3. In order for that to work, a special “object wrapper” that provides the extra functionality is created, and then is destroyed.

The “object wrappers” are different for each primitive type and are called: `String`, `Number`, `Boolean`, `Symbol` and `BigInt`. Thus, they provide different sets of methods.

For instance, there exists a string method `str.toUpperCase()` [↗](#) that returns a capitalized `str`.

Here's how it works:

```
let str = "Hello";

alert( str.toUpperCase() ); // HELLO
```

Simple, right? Here's what actually happens in `str.toUpperCase()`:

1. The string `str` is a primitive. So in the moment of accessing its property, a special object is created that knows the value of the string, and has useful methods, like `toUpperCase()`.
2. That method runs and returns a new string (shown by `alert()`).
3. The special object is destroyed, leaving the primitive `str` alone.

So primitives can provide methods, but they still remain lightweight.

The JavaScript engine highly optimizes this process. It may even skip the creation of the extra object at all. But it must still adhere to the specification and behave as if it creates one.

A number has methods of its own, for instance, `toFixed(n)` [↗](#) rounds the number to the given precision:

```
let n = 1.23456;

alert( n.toFixed(2) ); // 1.23
```

We'll see more specific methods in chapters [Numbers](#) and [Strings](#).

⚠ Constructors `String/Number/Boolean` are for internal use only

Some languages like Java allow us to explicitly create “wrapper objects” for primitives using a syntax like `new Number(1)` or `new Boolean(false)`.

In JavaScript, that's also possible for historical reasons, but highly **unrecommended**. Things will go crazy in several places.

For instance:

```
alert( typeof 0 ); // "number"

alert( typeof new Number(0) ); // "object!"
```

Objects are always truthy in `if`, so here the alert will show up:

```
let zero = new Number(0);

if (zero) { // zero is true, because it's an object
  alert( "zero is truthy!?" );
}
```

On the other hand, using the same functions `String/Number/Boolean` without `new` is totally fine and useful thing. They convert a value to the corresponding type: to a string, a number, or a boolean (primitive).

For example, this is entirely valid:

```
let num = Number("123"); // convert a string to number
```

⚠ `null/undefined` have no methods

The special primitives `null` and `undefined` are exceptions. They have no corresponding “wrapper objects” and provide no methods. In a sense, they are “the most primitive”.

An attempt to access a property of such value would give the error:

```
alert(null.test); // error
```

Summary

- Primitives except `null` and `undefined` provide many helpful methods. We will study those in the upcoming chapters.
- Formally, these methods work via temporary objects, but JavaScript engines are well tuned to optimize that internally, so they are not expensive to call.

✅ Tasks

Can I add a string property?

importance: 5

Consider the following code:

```
let str = "Hello";

str.test = 5;
```

```
alert(str.test);
```

What do you think, will it work? What will be shown?

[To solution](#)

Numbers

In modern JavaScript, there are two types of numbers:

1. Regular numbers in JavaScript are stored in 64-bit format [IEEE-754](#), also known as “double precision floating point numbers”. These are numbers that we’re using most of the time, and we’ll talk about them in this chapter.
2. BigInt numbers represent integers of arbitrary length. They are sometimes needed because a regular integer number can’t safely exceed $(2^{53}-1)$ or be less than $-(2^{53}-1)$, as we mentioned earlier in the chapter [Data types](#). As bigints are used in a few special areas, we devote them to a special chapter [BigInt](#).

So here we’ll talk about regular numbers. Let’s expand our knowledge of them.

More ways to write a number

Imagine we need to write 1 billion. The obvious way is:

```
let billion = 1000000000;
```

We also can use underscore `_` as the separator:

```
let billion = 1_000_000_000;
```

Here the underscore `_` plays the role of the “[syntactic sugar](#)”, it makes the number more readable. The JavaScript engine simply ignores `_` between digits, so it’s exactly the same one billion as above.

In real life though, we try to avoid writing long sequences of zeroes. We’re too lazy for that. We’ll try to write something like “1bn” for a billion or “7.3bn” for 7 billion 300 million. The same is true for most large numbers.

In JavaScript, we can shorten a number by appending the letter “e” to it and specifying the zeroes count:

```
let billion = 1e9; // 1 billion, literally: 1 and 9 zeroes  
  
alert( 7.3e9 ); // 7.3 billions (same as 7300000000 or 7_300_000_000)
```

In other words, `e` multiplies the number by `1` with the given zeroes count.

```
1e3 === 1 * 1000; // e3 means *1000  
1.23e6 === 1.23 * 1000000; // e6 means *1000000
```

Now let’s write something very small. Say, 1 microsecond (one-millionth of a second):

```
let mcs = 0.000001;
```

Just like before, using “e” can help. If we’d like to avoid writing the zeroes explicitly, we could write the same as:

```
let mcs = 1e-6; // five zeroes to the left from 1
```

If we count the zeroes in `0.000001`, there are 6 of them. So naturally it’s `1e-6`.

In other words, a negative number after “e” means a division by 1 with the given number of zeroes:

```
// -3 divides by 1 with 3 zeroes
1e-3 === 1 / 1000; // 0.001

// -6 divides by 1 with 6 zeroes
1.23e-6 === 1.23 / 1000000; // 0.00000123

// an example with a bigger number
1234e-2 === 1234 / 100; // 12.34, decimal point moves 2 times
```

Hex, binary and octal numbers

Hexadecimal [↗](#) numbers are widely used in JavaScript to represent colors, encode characters, and for many other things. So naturally, there exists a shorter way to write them: `0x` and then the number.

For instance:

```
alert( 0xff ); // 255
alert( 0xFF ); // 255 (the same, case doesn't matter)
```

Binary and octal numeral systems are rarely used, but also supported using the `0b` and `0o` prefixes:

```
let a = 0b11111111; // binary form of 255
let b = 0o377; // octal form of 255

alert( a == b ); // true, the same number 255 at both sides
```

There are only 3 numeral systems with such support. For other numeral systems, we should use the function `parseInt` (which we will see later in this chapter).

toString(base)

The method `num.toString(base)` returns a string representation of `num` in the numeral system with the given `base`.

For example:

```
let num = 255;

alert( num.toString(16) ); // ff
alert( num.toString(2) ); // 11111111
```

The `base` can vary from `2` to `36`. By default, it's `10`.

Common use cases for this are:

- **base=16** is used for hex colors, character encodings etc, digits can be `0..9` or `A..F`.
- **base=2** is mostly for debugging bitwise operations, digits can be `0` or `1`.
- **base=36** is the maximum, digits can be `0..9` or `A..Z`. The whole Latin alphabet is used to represent a number. A funny, but useful case for `36` is when we need to turn a long numeric identifier into something shorter, for example, to make a short url. Can simply represent it in the numeral system with base `36`:

```
alert( 123456.toString(36) ); // 2n9c
```

Two dots to call a method

Please note that two dots in `123456.toString(36)` is not a typo. If we want to call a method directly on a number, like `toString` in the example above, then we need to place two dots `..` after it.

If we placed a single dot: `123456.toString(36)`, then there would be an error, because JavaScript syntax implies the decimal part after the first dot. And if we place one more dot, then JavaScript knows that the decimal part is empty and now uses the method.

Also could write `(123456).toString(36)`.

Rounding

One of the most used operations when working with numbers is rounding.

There are several built-in functions for rounding:

`Math.floor`

Rounds down: `3.1` becomes `3`, and `-1.1` becomes `-2`.

`Math.ceil`

Rounds up: `3.1` becomes `4`, and `-1.1` becomes `-1`.

`Math.round`

Rounds to the nearest integer: `3.1` becomes `3`, `3.6` becomes `4`. In the middle cases `3.5` rounds up to `4`, and `-3.5` rounds up to `-3`.

`Math.trunc` (not supported by Internet Explorer)

Removes anything after the decimal point without rounding: `3.1` becomes `3`, `-1.1` becomes `-1`.

Here's the table to summarize the differences between them:

	<code>Math.floor</code>	<code>Math.ceil</code>	<code>Math.round</code>	<code>Math.trunc</code>
<code>3.1</code>	<code>3</code>	<code>4</code>	<code>3</code>	<code>3</code>
<code>3.5</code>	<code>3</code>	<code>4</code>	<code>4</code>	<code>3</code>
<code>3.6</code>	<code>3</code>	<code>4</code>	<code>4</code>	<code>3</code>
<code>-1.1</code>	<code>-2</code>	<code>-1</code>	<code>-1</code>	<code>-1</code>
<code>-1.5</code>	<code>-2</code>	<code>-1</code>	<code>-1</code>	<code>-1</code>
<code>-1.6</code>	<code>-2</code>	<code>-1</code>	<code>-2</code>	<code>-1</code>

These functions cover all of the possible ways to deal with the decimal part of a number. But what if we'd like to round the number to `n-th` digit after the decimal?

For instance, we have `1.2345` and want to round it to 2 digits, getting only `1.23`.

There are two ways to do so:

1. Multiply-and-divide.

For example, to round the number to the 2nd digit after the decimal, we can multiply the number by `100`, call the rounding function and then divide it back.

```
let num = 1.23456;
alert( Math.round(num * 100) / 100 ); // 1.23456 -> 123.456 -> 123 -> 1.23
```

2. The method `toFixed(n)` [↗](#) rounds the number to `n` digits after the point and returns a string representation of the result.

```
let num = 12.34;
alert( num.toFixed(1) ); // "12.3"
```

This rounds up or down to the nearest value, similar to `Math.round`:

```
let num = 12.36;
alert( num.toFixed(1) ); // "12.4"
```

Please note that the result of `toFixed` is a string. If the decimal part is shorter than required, zeroes are appended to the end:

```
let num = 12.34;
alert( num.toFixed(5) ); // "12.34000", added zeroes to make exactly 5 digits
```

How to round `6.35` the right way?

[To solution](#)

Repeat until the input is a number

importance: 5

Create a function `readNumber` which prompts for a number until the visitor enters a valid numeric value.

The resulting value must be returned as a number.

The visitor can also stop the process by entering an empty line or pressing “CANCEL”. In that case, the function should return `null`.

[Run the demo](#)

[Open a sandbox with tests.](#) ↗

[To solution](#)

An occasional infinite loop

importance: 4

This loop is infinite. It never ends. Why?

```
let i = 0;
while (i !== 10) {
  i += 0.2;
}
```

[To solution](#)

A random number from min to max

importance: 2

The built-in function `Math.random()` creates a random value from `0` to `1` (not including `1`).

Write the function `random(min, max)` to generate a random floating-point number from `min` to `max` (not including `max`).

Examples of its work:

```
alert( random(1, 5) ); // 1.2345623452
alert( random(1, 5) ); // 3.7894332423
alert( random(1, 5) ); // 4.3435234525
```

[To solution](#)

A random integer from min to max

importance: 2

Create a function `randomInteger(min, max)` that generates a random *integer* number from `min` to `max` including both `min` and `max` as possible values.

Any number from the interval `min..max` must appear with the same probability.

Examples of its work:

```
alert( randomInteger(1, 5) ); // 1
alert( randomInteger(1, 5) ); // 3
alert( randomInteger(1, 5) ); // 5
```

You can use the solution of the [previous task](#) as the base.

[To solution](#)

Strings

In JavaScript, the textual data is stored as strings. There is no separate type for a single character.

The internal format for strings is always [UTF-16](#), it is not tied to the page encoding.

Quotes

Let's recall the kinds of quotes.

Strings can be enclosed within either single quotes, double quotes or backticks:

```
let single = 'single-quoted';
let double = "double-quoted";

let backticks = `backticks`;
```

Single and double quotes are essentially the same. Backticks, however, allow us to embed any expression into the string, by wrapping it in `${...}`:

```
function sum(a, b) {
  return a + b;
}

alert(`1 + 2 = ${sum(1, 2)}.`); // 1 + 2 = 3.
```

Another advantage of using backticks is that they allow a string to span multiple lines:

```
let guestList = `Guests:
* John
* Pete
* Mary
`;

alert(guestList); // a list of guests, multiple lines
```

Looks natural, right? But single or double quotes do not work this way.

If we use them and try to use multiple lines, there'll be an error:

```
let guestList = "Guests: // Error: Unexpected token ILLEGAL
* John";
```

Single and double quotes come from ancient times of language creation, when the need for multiline strings was not taken into account. Backticks appeared much later and thus are more versatile.

Backticks also allow us to specify a “template function” before the first backtick. The syntax is: `func`string``. The function `func` is called automatically, receives the string and embedded expressions and can process them. This feature is called “tagged templates”, it's rarely seen, but you can read about it in the MDN: [Template literals](#).

Special characters

It is still possible to create multiline strings with single and double quotes by using a so-called “newline character”, written as `\n`, which denotes a line break:

```
let guestList = "Guests:\n * John\n * Pete\n * Mary";

alert(guestList); // a multiline list of guests, same as above
```

As a simpler example, these two lines are equal, just written differently:

```
let str1 = "Hello\nWorld"; // two lines using a "newline symbol"

// two lines using a normal newline and backticks
let str2 = `Hello
World`;

alert(str1 == str2); // true
```

There are other, less common special characters:

Character	Description
<code>\n</code>	New line
<code>\r</code>	In Windows text files a combination of two characters <code>\r\n</code> represents a new break, while on non-Windows OS it's just <code>\n</code> . That's for historical reasons, most Windows software also understands <code>\n</code> .
<code>\'</code> , <code>\"</code> , <code>\`</code>	Quotes
<code>\\</code>	Backslash
<code>\t</code>	Tab
<code>\b</code> , <code>\f</code> , <code>\v</code>	Backspace, Form Feed, Vertical Tab – mentioned for completeness, coming from old times, not used nowadays (you can forget them right now).

As you can see, all special characters start with a backslash character `\`. It is also called an “escape character”.

Because it's so special, if we need to show an actual backslash `\` within the string, we need to double it:

```
alert( `The backslash: \\` ); // The backslash: \
```

So-called “escaped” quotes `\'`, `\"`, `\`` are used to insert a quote into the same-quoted string.

For instance:

```
alert( 'I\'m the Walrus!' ); // I'm the Walrus!
```

As you can see, we have to prepend the inner quote by the backslash `\'`, because otherwise it would indicate the string end.

Of course, only the quotes that are the same as the enclosing ones need to be escaped. So, as a more elegant solution, we could switch to double quotes or backticks instead:

```
alert( "I'm the Walrus!" ); // I'm the Walrus!
```

Besides these special characters, there's also a special notation for Unicode codes `\u...`, it's rarely used and is covered in the optional chapter about [Unicode](#).

String length

The `length` property has the string length:

```
alert( `My\n`.length ); // 3
```

Note that `\n` is a single “special” character, so the length is indeed `3`.

`length` is a property

People with a background in some other languages sometimes mistype by calling `str.length()` instead of just `str.length`. That doesn't work.

Please note that `str.length` is a numeric property, not a function. There is no need to add parenthesis after it. Not `.length()`, but `.length`.

Accessing characters

To get a character at position `pos`, use square brackets `[pos]` or call the method `str.at(pos)`. The first character starts from the zero position:

```
let str = `Hello`;

// the first character
alert( str[0] ); // H
alert( str.at(0) ); // H

// the last character
alert( str[str.length - 1] ); // o
alert( str.at(-1) );
```

As you can see, the `.at(pos)` method has a benefit of allowing negative position. If `pos` is negative, then it's counted from the end of the string.

So `.at(-1)` means the last character, and `.at(-2)` is the one before it, etc.

The square brackets always return `undefined` for negative indexes, for instance:

```
let str = `Hello`;

alert( str[-2] ); // undefined
alert( str.at(-2) ); // l
```

We can also iterate over characters using `for...of`:

```
for (let char of "Hello") {
  alert(char); // H,e,l,l,o (char becomes "H", then "e", then "l" etc)
}
```

Strings are immutable

Strings can't be changed in JavaScript. It is impossible to change a character.

Let's try it to show that it doesn't work:

```
let str = 'Hi';

str[0] = 'h'; // error
alert( str[0] ); // doesn't work
```

The usual workaround is to create a whole new string and assign it to `str` instead of the old one.

For instance:

```
let str = 'Hi';

str = 'h' + str[1]; // replace the string

alert( str ); // hi
```

In the following sections we'll see more examples of this.

Changing the case

Methods `toLowerCase()` and `toUpperCase()` change the case:

```
alert( 'Interface'.toUpperCase() ); // INTERFACE
alert( 'Interface'.toLowerCase() ); // interface
```

Or, if we want a single character lowercased:

```
alert( 'Interface'[0].toLowerCase() ); // 'i'
```

Searching for a substring

There are multiple ways to look for a substring within a string.

str.indexOf

The first method is `str.indexOf(substr, pos)` [↗](#).

It looks for the `substr` in `str`, starting from the given position `pos`, and returns the position where the match was found or `-1` if nothing can be found.

For instance:

```
let str = 'Widget with id';

alert( str.indexOf('Widget') ); // 0, because 'Widget' is found at the beginning
alert( str.indexOf('widget') ); // -1, not found, the search is case-sensitive

alert( str.indexOf("id") ); // 1, "id" is found at the position 1 (..idget with id)
```

The optional second parameter allows us to start searching from a given position.

For instance, the first occurrence of `"id"` is at position `1`. To look for the next occurrence, let's start the search from position `2`:

```
let str = 'Widget with id';

alert( str.indexOf('id', 2) ) // 12
```

If we're interested in all occurrences, we can run `indexOf` in a loop. Every new call is made with the position after the previous match:

```
let str = 'As sly as a fox, as strong as an ox';

let target = 'as'; // let's look for it

let pos = 0;
while (true) {
  let foundPos = str.indexOf(target, pos);
  if (foundPos == -1) break;

  alert( `Found at ${foundPos}` );
  pos = foundPos + 1; // continue the search from the next position
}
```

The same algorithm can be layed out shorter:

```
let str = "As sly as a fox, as strong as an ox";
let target = "as";

let pos = -1;
while ((pos = str.indexOf(target, pos + 1)) != -1) {
  alert( pos );
}
```

str.lastIndexOf(substr, position)

There is also a similar method `str.lastIndexOf(substr, position)` [↗](#) that searches from the end of a string to its beginning.

It would list the occurrences in the reverse order.

There is a slight inconvenience with `indexOf` in the `if` test. We can't put it in the `if` like this:

```
let str = "Widget with id";

if (str.indexOf("Widget")) {
  alert("We found it"); // doesn't work!
}
```

The `alert` in the example above doesn't show because `str.indexOf("Widget")` returns `0` (meaning that it found the match at the starting position). Right, but `if` considers `0` to be `false`.

So, we should actually check for `-1`, like this:

```
let str = "Widget with id";

if (str.indexOf("Widget") !== -1) {
  alert("We found it"); // works now!
}
```

includes, startsWith, endsWith

The more modern method `str.includes(substr, pos)` [↗](#) returns `true/false` depending on whether `str` contains `substr` within.

It's the right choice if we need to test for the match, but don't need its position:

```
alert( "Widget with id".includes("Widget") ); // true

alert( "Hello".includes("Bye") ); // false
```

The optional second argument of `str.includes` is the position to start searching from:

```
alert( "Widget".includes("id") ); // true
alert( "Widget".includes("id", 3) ); // false, from position 3 there is no "id"
```

The methods `str.startsWith` [↗](#) and `str.endsWith` [↗](#) do exactly what they say:

```
alert( "Widget".startsWith("Wid") ); // true, "Widget" starts with "Wid"
alert( "Widget".endsWith("get") ); // true, "Widget" ends with "get"
```

Getting a substring

There are 3 methods in JavaScript to get a substring: `substring`, `substr` and `slice`.

`str.slice(start [, end])`

Returns the part of the string from `start` to (but not including) `end`.

For instance:

```
let str = "stringify";
alert( str.slice(0, 5) ); // 'strin', the substring from 0 to 5 (not including 5)
alert( str.slice(0, 1) ); // 's', from 0 to 1, but not including 1, so only character at 0
```

If there is no second argument, then `slice` goes till the end of the string:

```
let str = "stringify";
alert( str.slice(2) ); // 'ringify', from the 2nd position till the end
```

Negative values for `start/end` are also possible. They mean the position is counted from the string end:

```
let str = "stringify";

// start at the 4th position from the right, end at the 1st from the right
alert( str.slice(-4, -1) ); // 'gif'
```

str.substring(start [, end])

Returns the part of the string *between* `start` and `end` (not including `end`).

This is almost the same as `slice`, but it allows `start` to be greater than `end` (in this case it simply swaps `start` and `end` values).

For instance:

```
let str = "stringify";

// these are same for substring
alert( str.substring(2, 6) ); // "ring"
alert( str.substring(6, 2) ); // "ring"

// ...but not for slice:
alert( str.slice(2, 6) ); // "ring" (the same)
alert( str.slice(6, 2) ); // "" (an empty string)
```

Negative arguments are (unlike `slice`) not supported, they are treated as `0`.

str.substr(start [, length])

Returns the part of the string from `start`, with the given `length`.

In contrast with the previous methods, this one allows us to specify the `length` instead of the ending position:

```
let str = "stringify";
alert( str.substr(2, 4) ); // 'ring', from the 2nd position get 4 characters
```

The first argument may be negative, to count from the end:

```
let str = "stringify";
alert( str.substr(-4, 2) ); // 'gi', from the 4th position get 2 characters
```

This method resides in the [Annex B](#) of the language specification. It means that only browser-hosted Javascript engines should support it, and it's not recommended to use it. In practice, it's supported everywhere.

Let's recap these methods to avoid any confusion:

method	selects...	negatives
<code>slice(start, end)</code>	from <code>start</code> to <code>end</code> (not including <code>end</code>)	allows negatives
<code>substring(start, end)</code>	between <code>start</code> and <code>end</code> (not including <code>end</code>)	negative values mean <code>0</code>
<code>substr(start, length)</code>	from <code>start</code> get <code>length</code> characters	allows negative <code>start</code>

Which one to choose?

All of them can do the job. Formally, `substr` has a minor drawback: it is described not in the core JavaScript specification, but in Annex B, which covers browser-only features that exist mainly for historical reasons. So, non-browser environments may fail to support it. But in practice it works everywhere.

Of the other two variants, `slice` is a little bit more flexible, it allows negative arguments and shorter to write.

So, for practical use it's enough to remember only `slice`.

Comparing strings

As we know from the chapter [Comparisons](#), strings are compared character-by-character in alphabetical order.

Although, there are some oddities.

1. A lowercase letter is always greater than the uppercase:

```
alert( 'a' > 'Z' ); // true
```

2. Letters with diacritical marks are “out of order”:

```
alert( 'Österreich' > 'Zealand' ); // true
```

This may lead to strange results if we sort these country names. Usually people would expect `Zealand` to come after `Österreich` in the list.

To understand what happens, we should be aware that strings in Javascript are encoded using [UTF-16](#). That is: each character has a corresponding numeric code.

There are special methods that allow to get the character for the code and back:

`str.codePointAt(pos)`

Returns a decimal number representing the code for the character at position `pos`:

```
// different case letters have different codes
alert( "Z".codePointAt(0) ); // 90
alert( "z".codePointAt(0) ); // 122
alert( "z".codePointAt(0).toString(16) ); // 7a (if we need a hexadecimal value)
```

`String.fromCharCode(code)`

Creates a character by its numeric `code`

```
alert( String.fromCharCode(90) ); // Z
alert( String.fromCharCode(0x5a) ); // Z (we can also use a hex value as an argument)
```

Now let's see the characters with codes `65..220` (the latin alphabet and a little bit extra) by making a string of them:

```
let str = '';

for (let i = 65; i <= 220; i++) {
  str += String.fromCharCode(i);
}
alert( str );
// Output:
// ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~©000(
// ¡¢£¥¦§¨ª«¬®¯°±²³´µ¶·¸¹º»¼½¾¿ÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖ×ØÙÚÛÜ
```

See? Capital characters go first, then a few special ones, then lowercase characters, and `Ö` near the end of the output.

Now it becomes obvious why `a > Z`.

The characters are compared by their numeric code. The greater code means that the character is greater. The code for `a` (97) is greater than the code for `Z` (90).

- All lowercase letters go after uppercase letters because their codes are greater.
- Some letters like `Ö` stand apart from the main alphabet. Here, its code is greater than anything from `a` to `z`.

Correct comparisons

The “right” algorithm to do string comparisons is more complex than it may seem, because alphabets are different for different languages.

So, the browser needs to know the language to compare.

Luckily, modern browsers support the internationalization standard [ECMA-402](#).

It provides a special method to compare strings in different languages, following their rules.

The call `str.localeCompare(str2)` [↗](#) returns an integer indicating whether `str` is less, equal or greater than `str2` according to the language rules:

- Returns a negative number if `str` is less than `str2`.
- Returns a positive number if `str` is greater than `str2`.
- Returns `0` if they are equivalent.

For instance:

```
alert( 'Österreich'.localeCompare('Zealand') ); // -1
```

This method actually has two additional arguments specified in [the documentation](#) [↗](#), which allows it to specify the language (by default taken from the environment, letter order depends on the language) and setup additional rules like case sensitivity or should "a" and "á" be treated as the same etc.

Summary

- There are 3 types of quotes. Backticks allow a string to span multiple lines and embed expressions `${...}`.
- We can use special characters, such as a line break `\n`.
- To get a character, use: `[]` or `at` method.
- To get a substring, use: `slice` or `substring`.
- To lowercase/uppercase a string, use: `toLowerCase/toUpperCase`.
- To look for a substring, use: `indexOf`, or `includes/startsWith/endsWith` for simple checks.
- To compare strings according to the language, use: `localeCompare`, otherwise they are compared by character codes.

There are several other helpful methods in strings:

- `str.trim()` – removes ("trims") spaces from the beginning and end of the string.
- `str.repeat(n)` – repeats the string `n` times.
- ...and more to be found in the [manual](#) [↗](#).

Strings also have methods for doing search/replace with regular expressions. But that's big topic, so it's explained in a separate tutorial section [Regular expressions](#).

Also, as of now it's important to know that strings are based on Unicode encoding, and hence there're issues with comparisons. There's more about Unicode in the chapter [Unicode, String internals](#).

✓ Tasks

Uppercase the first character

importance: 5

Write a function `ucFirst(str)` that returns the string `str` with the uppercased first character, for instance:

```
ucFirst("john") == "John";
```

[Open a sandbox with tests.](#) [↗](#)

[To solution](#)

Check for spam

importance: 5

Write a function `checkSpam(str)` that returns `true` if `str` contains 'viagra' or 'XXX', otherwise `false`.

The function must be case-insensitive:

```
checkSpam('buy ViAgRA now') == true
checkSpam('free xxxxx') == true
```

```
checkSpam("innocent rabbit") == false
```

[Open a sandbox with tests.](#)

[To solution](#)

Truncate the text

importance: 5

Create a function `truncate(str, maxLength)` that checks the length of the `str` and, if it exceeds `maxLength` – replaces the end of `str` with the ellipsis character `"..."`, to make its length equal to `maxLength`.

The result of the function should be the truncated (if needed) string.

For instance:

```
truncate("What I'd like to tell on this topic is:", 20) == "What I'd like to te..."
truncate("Hi everyone!", 20) == "Hi everyone!"
```

[Open a sandbox with tests.](#)

[To solution](#)

Extract the money

importance: 4

We have a cost in the form `"$120"`. That is: the dollar sign goes first, and then the number.

Create a function `extractCurrencyValue(str)` that would extract the numeric value from such string and return it.

The example:

```
alert( extractCurrencyValue('$120') === 120 ); // true
```

[Open a sandbox with tests.](#)

[To solution](#)

Arrays

Objects allow you to store keyed collections of values. That's fine.

But quite often we find that we need an *ordered collection*, where we have a 1st, a 2nd, a 3rd element and so on. For example, we need that to store a list of something: users, goods, HTML elements etc.

It is not convenient to use an object here, because it provides no methods to manage the order of elements. We can't insert a new property "between" the existing ones. Objects are just not meant for such use.

There exists a special data structure named `Array`, to store ordered collections.

Declaration

There are two syntaxes for creating an empty array:

```
let arr = new Array();
let arr = [];
```

Almost all the time, the second syntax is used. We can supply initial elements in the brackets:

```
let fruits = ["Apple", "Orange", "Plum"];
```

Array elements are numbered, starting with zero.

We can get an element by its number in square brackets:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits[0] ); // Apple
alert( fruits[1] ); // Orange
alert( fruits[2] ); // Plum
```

We can replace an element:

```
fruits[2] = 'Pear'; // now ["Apple", "Orange", "Pear"]
```

...Or add a new one to the array:

```
fruits[3] = 'Lemon'; // now ["Apple", "Orange", "Pear", "Lemon"]
```

The total count of the elements in the array is its `length`:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits.length ); // 3
```

We can also use `alert` to show the whole array.

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits ); // Apple,Orange,Plum
```

An array can store elements of any type.

For instance:

```
// mix of values
let arr = [ 'Apple', { name: 'John' }, true, function() { alert('hello'); } ];

// get the object at index 1 and then show its name
alert( arr[1].name ); // John

// get the function at index 3 and run it
arr[3](); // hello
```

Trailing comma

An array, just like an object, may end with a comma:

```
let fruits = [
  "Apple",
  "Orange",
  "Plum",
];
```

The “trailing comma” style makes it easier to insert/remove items, because all lines become alike.

Get last elements with “at”

A recent addition

This is a recent addition to the language. Old browsers may need [polyfills](#).

Let's say we want the last element of the array.

Some programming languages allow the use of negative indexes for the same purpose, like `fruits[-1]`.

Although, in JavaScript it won't work. The result will be `undefined`, because the index in square brackets is treated literally.

We can explicitly calculate the last element index and then access it: `fruits[fruits.length - 1]`.

```
let fruits = ["Apple", "Orange", "Plum"];  
  
alert( fruits[fruits.length-1] ); // Plum
```

A bit cumbersome, isn't it? We need to write the variable name twice.

Luckily, there's a shorter syntax: `fruits.at(-1)`:

```
let fruits = ["Apple", "Orange", "Plum"];  
  
// same as fruits[fruits.length-1]  
alert( fruits.at(-1) ); // Plum
```

In other words, `arr.at(i)`:

- is exactly the same as `arr[i]`, if `i >= 0`.
- for negative values of `i`, it steps back from the end of the array.

Methods `pop`/`push`, `shift`/`unshift`

A [queue](#) is one of the most common uses of an array. In computer science, this means an ordered collection of elements which supports two operations:

- `push` appends an element to the end.
- `shift` get an element from the beginning, advancing the queue, so that the 2nd element becomes the 1st.



Arrays support both operations.

In practice we need it very often. For example, a queue of messages that need to be shown on-screen.

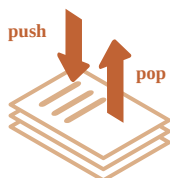
There's another use case for arrays – the data structure named [stack](#).

It supports two operations:

- `push` adds an element to the end.
- `pop` takes an element from the end.

So new elements are added or taken always from the “end”.

A stack is usually illustrated as a pack of cards: new cards are added to the top or taken from the top:



For stacks, the latest pushed item is received first, that's also called LIFO (Last-In-First-Out) principle. For queues, we have FIFO (First-In-First-Out).

Arrays in JavaScript can work both as a queue and as a stack. They allow you to add/remove elements, both to/from the beginning or the end.

In computer science, the data structure that allows this, is called [deque](#) .

Methods that work with the end of the array:

pop

Extracts the last element of the array and returns it:

```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.pop() ); // remove "Pear" and alert it

alert( fruits ); // Apple, Orange
```

Both `fruits.pop()` and `fruits.at(-1)` return the last element of the array, but `fruits.pop()` also modifies the array by removing it.

push

Append the element to the end of the array:

```
let fruits = ["Apple", "Orange"];

fruits.push("Pear");

alert( fruits ); // Apple, Orange, Pear
```

The call `fruits.push(...)` is equal to `fruits[fruits.length] = ...`.

Methods that work with the beginning of the array:

shift

Extracts the first element of the array and returns it:

```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.shift() ); // remove Apple and alert it

alert( fruits ); // Orange, Pear
```

unshift

Add the element to the beginning of the array:

```
let fruits = ["Orange", "Pear"];

fruits.unshift('Apple');

alert( fruits ); // Apple, Orange, Pear
```

Methods `push` and `unshift` can add multiple elements at once:

```
let fruits = ["Apple"];

fruits.push("Orange", "Peach");
fruits.unshift("Pineapple", "Lemon");

// ["Pineapple", "Lemon", "Apple", "Orange", "Peach"]
alert( fruits );
```

Internals

An array is a special kind of object. The square brackets used to access a property `arr[0]` actually come from the object syntax. That's essentially the same as `obj[key]`, where `arr` is the object, while numbers are used as keys.

They extend objects providing special methods to work with ordered collections of data and also the `length` property. But at the core it's still an object.

Remember, there are only eight basic data types in JavaScript (see the [Data types](#) chapter for more info). Array is an object and thus behaves like an object.

For instance, it is copied by reference:

```
let fruits = ["Banana"]

let arr = fruits; // copy by reference (two variables reference the same array)

alert( arr === fruits ); // true

arr.push("Pear"); // modify the array by reference

alert( fruits ); // Banana, Pear - 2 items now
```

...But what makes arrays really special is their internal representation. The engine tries to store its elements in the contiguous memory area, one after another, just as depicted on the illustrations in this chapter, and there are other optimizations as well, to make arrays work really fast.

But they all break if we quit working with an array as with an “ordered collection” and start working with it as if it were a regular object.

For instance, technically we can do this:

```
let fruits = []; // make an array

fruits[99999] = 5; // assign a property with the index far greater than its length

fruits.age = 25; // create a property with an arbitrary name
```

That's possible, because arrays are objects at their base. We can add any properties to them.

But the engine will see that we're working with the array as with a regular object. Array-specific optimizations are not suited for such cases and will be turned off, their benefits disappear.

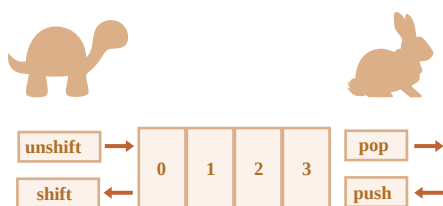
The ways to misuse an array:

- Add a non-numeric property like `arr.test = 5`.
- Make holes, like: add `arr[0]` and then `arr[1000]` (and nothing between them).
- Fill the array in the reverse order, like `arr[1000]`, `arr[999]` and so on.

Please think of arrays as special structures to work with the *ordered data*. They provide special methods for that. Arrays are carefully tuned inside JavaScript engines to work with contiguous ordered data, please use them this way. And if you need arbitrary keys, chances are high that you actually require a regular object `{}`.

Performance

Methods `push/pop` run fast, while `shift/unshift` are slow.



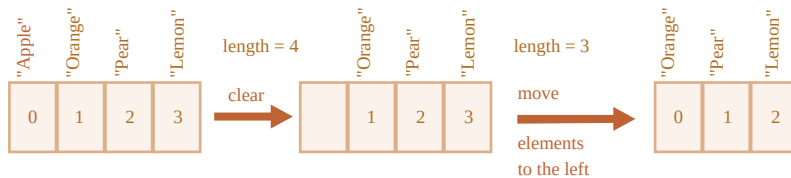
Why is it faster to work with the end of an array than with its beginning? Let's see what happens during the execution:

```
fruits.shift(); // take 1 element from the start
```

It's not enough to take and remove the element with the index `0`. Other elements need to be renumbered as well.

The `shift` operation must do 3 things:

1. Remove the element with the index `0`.
2. Move all elements to the left, renumber them from the index `1` to `0`, from `2` to `1` and so on.
3. Update the `length` property.



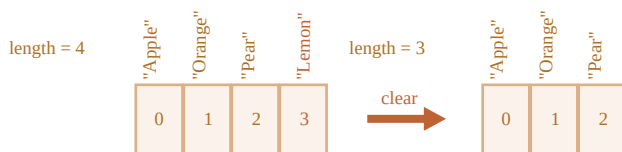
The more elements in the array, the more time to move them, more in-memory operations.

The similar thing happens with `unshift`: to add an element to the beginning of the array, we need first to move existing elements to the right, increasing their indexes.

And what's with `push/pop`? They do not need to move anything. To extract an element from the end, the `pop` method cleans the index and shortens `length`.

The actions for the `pop` operation:

```
fruits.pop(); // take 1 element from the end
```



The `pop` method does not need to move anything, because other elements keep their indexes. That's why it's blazingly fast.

The similar thing with the `push` method.

Loops

One of the oldest ways to cycle array items is the `for` loop over indexes:

```
let arr = ["Apple", "Orange", "Pear"];

for (let i = 0; i < arr.length; i++) {
  alert( arr[i] );
}
```

But for arrays there is another form of loop, `for...of`:

```
let fruits = ["Apple", "Orange", "Plum"];

// iterates over array elements
for (let fruit of fruits) {
  alert( fruit );
}
```

The `for...of` doesn't give access to the number of the current element, just its value, but in most cases that's enough. And it's shorter.

Technically, because arrays are objects, it is also possible to use `for...in`:

```
let arr = ["Apple", "Orange", "Pear"];

for (let key in arr) {
  alert( arr[key] ); // Apple, Orange, Pear
}
```

But that's actually a bad idea. There are potential problems with it:

1. The loop `for...in` iterates over *all properties*, not only the numeric ones.

There are so-called “array-like” objects in the browser and in other environments, that *look like arrays*. That is, they have `length` and indexes properties, but they may also have other non-numeric properties and methods, which we usually don't need. The `for...in` loop will list them though. So if we need to work with array-like objects, then these “extra” properties can become a problem.

2. The `for...in` loop is optimized for generic objects, not arrays, and thus is 10-100 times slower. Of course, it's still very fast. The speedup may only matter in bottlenecks. But still we should be aware of the difference.

Generally, we shouldn't use `for...in` for arrays.

A word about “length”

The `length` property automatically updates when we modify the array. To be precise, it is actually not the count of values in the array, but the greatest numeric index plus one.

For instance, a single element with a large index gives a big length:

```
let fruits = [];
fruits[123] = "Apple";

alert( fruits.length ); // 124
```

Note that we usually don't use arrays like that.

Another interesting thing about the `length` property is that it's writable.

If we increase it manually, nothing interesting happens. But if we decrease it, the array is truncated. The process is irreversible, here's the example:

```
let arr = [1, 2, 3, 4, 5];

arr.length = 2; // truncate to 2 elements
alert( arr ); // [1, 2]

arr.length = 5; // return length back
alert( arr[3] ); // undefined: the values do not return
```

So, the simplest way to clear the array is: `arr.length = 0;`

new Array()

There is one more syntax to create an array:

```
let arr = new Array("Apple", "Pear", "etc");
```

It's rarely used, because square brackets `[]` are shorter. Also, there's a tricky feature with it.

If `new Array` is called with a single argument which is a number, then it creates an array *without items, but with the given length*.

Let's see how one can shoot themselves in the foot:

```
let arr = new Array(2); // will it create an array of [2] ?

alert( arr[0] ); // undefined! no elements.

alert( arr.length ); // length 2
```

To avoid such surprises, we usually use square brackets, unless we really know what we're doing.

Multidimensional arrays

Arrays can have items that are also arrays. We can use it for multidimensional arrays, for example to store matrices:

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

alert( matrix[0][1] ); // 2, the second value of the first inner array
```

toString

Arrays have their own implementation of `toString` method that returns a comma-separated list of elements.

For instance:

```
let arr = [1, 2, 3];

alert( arr ); // 1,2,3
alert( String(arr) === '1,2,3' ); // true
```

Also, let's try this:

```
alert( [] + 1 ); // "1"
alert( [1] + 1 ); // "11"
alert( [1,2] + 1 ); // "1,21"
```

Arrays do not have `Symbol.toPrimitive`, neither a viable `valueOf`, they implement only `toString` conversion, so here `[]` becomes an empty string, `[1]` becomes `"1"` and `[1,2]` becomes `"1,2"`.

When the binary plus `+` operator adds something to a string, it converts it to a string as well, so the next step looks like this:

```
alert( "" + 1 ); // "1"
alert( "1" + 1 ); // "11"
alert( "1,2" + 1 ); // "1,21"
```

Don't compare arrays with ==

Arrays in JavaScript, unlike some other programming languages, shouldn't be compared with operator `==`.

This operator has no special treatment for arrays, it works with them as with any objects.

Let's recall the rules:

- Two objects are equal `==` only if they're references to the same object.
- If one of the arguments of `==` is an object, and the other one is a primitive, then the object gets converted to primitive, as explained in the chapter [Object to primitive conversion](#).
- ...With an exception of `null` and `undefined` that equal `==` each other and nothing else.

The strict comparison `===` is even simpler, as it doesn't convert types.

So, if we compare arrays with `===`, they are never the same, unless we compare two variables that reference exactly the same array.

For example:

```
alert( [] == [] ); // false
alert( [0] == [0] ); // false
```

These arrays are technically different objects. So they aren't equal. The `===` operator doesn't do item-by-item comparison.

Comparison with primitives may give seemingly strange results as well:

```
alert( 0 == [] ); // true
alert( '0' == [] ); // false
```

Here, in both cases, we compare a primitive with an array object. So the array `[]` gets converted to primitive for the purpose of comparison and becomes an empty string `''`.

Then the comparison process goes on with the primitives, as described in the chapter [Type Conversions](#):

```
// after [] was converted to ''
alert( 0 == '' ); // true, as '' becomes converted to number 0

alert( '0' == '' ); // false, no type conversion, different strings
```

So, how to compare arrays?

That's simple: don't use the `===` operator. Instead, compare them item-by-item in a loop or using iteration methods explained in the next chapter.

Summary

Array is a special kind of object, suited to storing and managing ordered data items.

The declaration:

```
// square brackets (usual)
let arr = [item1, item2...];

// new Array (exceptionally rare)
let arr = new Array(item1, item2...);
```

The call to `new Array(number)` creates an array with the given length, but without elements.

- The `length` property is the array length or, to be precise, its last numeric index plus one. It is auto-adjusted by array methods.
- If we shorten `length` manually, the array is truncated.

Getting the elements:

- we can get element by its index, like `arr[0]`
- also we can use `at(i)` method that allows negative indexes. For negative values of `i`, it steps back from the end of the array. If `i >= 0`, it works same as `arr[i]`.

We can use an array as a deque with the following operations:

- `push(...items)` adds `items` to the end.
- `pop()` removes the element from the end and returns it.
- `shift()` removes the element from the beginning and returns it.
- `unshift(...items)` adds `items` to the beginning.

To loop over the elements of the array:

- `for (let i=0; i<arr.length; i++)` – works fastest, old-browser-compatible.

- `for (let item of arr)` – the modern syntax for items only,
- `for (let i in arr)` – never use.

To compare arrays, don't use the `==` operator (as well as `>`, `<` and others), as they have no special treatment for arrays. They handle them as any objects, and it's not what we usually want.

Instead you can use `for...of` loop to compare arrays item-by-item.

We will continue with arrays and study more methods to add, remove, extract elements and sort arrays in the next chapter [Array methods](#).

✓ Tasks

Is array copied?

importance: 3

What is this code going to show?

```
let fruits = ["Apples", "Pear", "Orange"];

// push a new value into the "copy"
let shoppingCart = fruits;
shoppingCart.push("Banana");

// what's in fruits?
alert( fruits.length ); // ?
```

[To solution](#)

Array operations.

importance: 5

Let's try 5 array operations.

1. Create an array `styles` with items "Jazz" and "Blues".
2. Append "Rock-n-Roll" to the end.
3. Replace the value in the middle with "Classics". Your code for finding the middle value should work for any arrays with odd length.
4. Strip off the first value of the array and show it.
5. Prepend `Rap` and `Reggae` to the array.

The array in the process:

```
Jazz, Blues
Jazz, Blues, Rock-n-Roll
Jazz, Classics, Rock-n-Roll
Classics, Rock-n-Roll
Rap, Reggae, Classics, Rock-n-Roll
```

[To solution](#)

Calling in an array context

importance: 5

What is the result? Why?

```
let arr = ["a", "b"];

arr.push(function() {
  alert( this );
});

arr[2](); // ?
```

[To solution](#)

Sum input numbers

importance: 4

Write the function `sumInput()` that:

- Asks the user for values using `prompt` and stores the values in the array.
- Finishes asking when the user enters a non-numeric value, an empty string, or presses "Cancel".
- Calculates and returns the sum of array items.

P.S. A zero `0` is a valid number, please don't stop the input on zero.

[Run the demo](#)

[To solution](#)

A maximal subarray

importance: 2

The input is an array of numbers, e.g. `arr = [1, -2, 3, 4, -9, 6]`.

The task is: find the contiguous subarray of `arr` with the maximal sum of items.

Write the function `getMaxSubSum(arr)` that will return that sum.

For instance:

```
getMaxSubSum([-1, 2, 3, -9]) == 5 (the sum of highlighted items)
getMaxSubSum([2, -1, 2, 3, -9]) == 6
getMaxSubSum([-1, 2, 3, -9, 11]) == 11
getMaxSubSum([-2, -1, 1, 2]) == 3
getMaxSubSum([100, -9, 2, -3, 5]) == 100
getMaxSubSum([1, 2, 3]) == 6 (take all)
```

If all items are negative, it means that we take none (the subarray is empty), so the sum is zero:

```
getMaxSubSum([-1, -2, -3]) = 0
```

Please try to think of a fast solution: $O(n^2)$ or even $O(n)$ if you can.

[Open a sandbox with tests.](#)

[To solution](#)

Array methods

Arrays provide a lot of methods. To make things easier, in this chapter, they are split into groups.

Add/remove items

We already know methods that add and remove items from the beginning or the end:

- `arr.push(...items)` – adds items to the end,
- `arr.pop()` – extracts an item from the end,
- `arr.shift()` – extracts an item from the beginning,
- `arr.unshift(...items)` – adds items to the beginning.

Here are a few others.

splice

How to delete an element from the array?

The arrays are objects, so we can try to use `delete`:

```
let arr = ["I", "go", "home"];

delete arr[1]; // remove "go"

alert( arr[1] ); // undefined

// now arr = ["I", , "home"];
alert( arr.length ); // 3
```

The element was removed, but the array still has 3 elements, we can see that `arr.length == 3`.

That's natural, because `delete obj.key` removes a value by the `key`. It's all it does. Fine for objects. But for arrays we usually want the rest of the elements to shift and occupy the freed place. We expect to have a shorter array now.

So, special methods should be used.

The `arr.splice` [↗](#) method is a Swiss army knife for arrays. It can do everything: insert, remove and replace elements.

The syntax is:

```
arr.splice(start[, deleteCount, elem1, ..., elemN])
```

It modifies `arr` starting from the index `start`: removes `deleteCount` elements and then inserts `elem1, ..., elemN` at their place. Returns the array of removed elements.

This method is easy to grasp by examples.

Let's start with the deletion:

```
let arr = ["I", "study", "JavaScript"];

arr.splice(1, 1); // from index 1 remove 1 element

alert( arr ); // ["I", "JavaScript"]
```

Easy, right? Starting from the index `1` it removed `1` element.

In the next example, we remove 3 elements and replace them with the other two:

```
let arr = ["I", "study", "JavaScript", "right", "now"];

// remove 3 first elements and replace them with another
arr.splice(0, 3, "Let's", "dance");

alert( arr ) // now ["Let's", "dance", "right", "now"]
```

Here we can see that `splice` returns the array of removed elements:

```
let arr = ["I", "study", "JavaScript", "right", "now"];

// remove 2 first elements
let removed = arr.splice(0, 2);

alert( removed ); // "I", "study" <-- array of removed elements
```

The `splice` method is also able to insert the elements without any removals. For that, we need to set `deleteCount` to `0`:

```
let arr = ["I", "study", "JavaScript"];

// from index 2
```

```
// delete 0
// then insert "complex" and "language"
arr.splice(2, 0, "complex", "language");

alert( arr ); // "I", "study", "complex", "language", "JavaScript"
```

i Negative indexes allowed

Here and in other array methods, negative indexes are allowed. They specify the position from the end of the array, like here:

```
let arr = [1, 2, 5];

// from index -1 (one step from the end)
// delete 0 elements,
// then insert 3 and 4
arr.splice(-1, 0, 3, 4);

alert( arr ); // 1,2,3,4,5
```

slice

The method `arr.slice` [↗](#) is much simpler than the similar-looking `arr.splice`.

The syntax is:

```
arr.slice([start], [end])
```

It returns a new array copying to it all items from index `start` to `end` (not including `end`). Both `start` and `end` can be negative, in that case position from array end is assumed.

It's similar to a string method `str.slice`, but instead of substrings, it makes subarrays.

For instance:

```
let arr = ["t", "e", "s", "t"];

alert( arr.slice(1, 3) ); // e,s (copy from 1 to 3)

alert( arr.slice(-2) ); // s,t (copy from -2 till the end)
```

We can also call it without arguments: `arr.slice()` creates a copy of `arr`. That's often used to obtain a copy for further transformations that should not affect the original array.

concat

The method `arr.concat` [↗](#) creates a new array that includes values from other arrays and additional items.

The syntax is:

```
arr.concat(arg1, arg2...)
```

It accepts any number of arguments – either arrays or values.

The result is a new array containing items from `arr`, then `arg1`, `arg2` etc.

If an argument `argN` is an array, then all its elements are copied. Otherwise, the argument itself is copied.

For instance:

```
let arr = [1, 2];

// create an array from: arr and [3,4]
alert( arr.concat([3, 4]) ); // 1,2,3,4

// create an array from: arr and [3,4] and [5,6]
alert( arr.concat([3, 4], [5, 6]) ); // 1,2,3,4,5,6
```

```
// create an array from: arr and [3,4], then add values 5 and 6
alert( arr.concat([3, 4], 5, 6) ); // 1,2,3,4,5,6
```

Normally, it only copies elements from arrays. Other objects, even if they look like arrays, are added as a whole:

```
let arr = [1, 2];

let arrayLike = {
  0: "something",
  length: 1
};

alert( arr.concat(arrayLike) ); // 1,2,[object Object]
```

...But if an array-like object has a special `Symbol.isConcatSpreadable` property, then it's treated as an array by `concat`: its elements are added instead:

```
let arr = [1, 2];

let arrayLike = {
  0: "something",
  1: "else",
  [Symbol.isConcatSpreadable]: true,
  length: 2
};

alert( arr.concat(arrayLike) ); // 1,2,something,else
```

Iterate: `forEach`

The `arr.forEach` [↗](#) method allows to run a function for every element of the array.

The syntax:

```
arr.forEach(function(item, index, array) {
  // ... do something with an item
});
```

For instance, this shows each element of the array:

```
// for each element call alert
["Bilbo", "Gandalf", "Nazgul"].forEach(alert);
```

And this code is more elaborate about their positions in the target array:

```
["Bilbo", "Gandalf", "Nazgul"].forEach((item, index, array) => {
  alert(`${item} is at index ${index} in ${array}`);
});
```

The result of the function (if it returns any) is thrown away and ignored.

Searching in array

Now let's cover methods that search in an array.

`indexOf/lastIndexOf` and `includes`

The methods `arr.indexOf` [↗](#) and `arr.includes` [↗](#) have the similar syntax and do essentially the same as their string counterparts, but operate on items instead of characters:

- `arr.indexOf(item, from)` – looks for `item` starting from index `from`, and returns the index where it was found, otherwise `-1`.
- `arr.includes(item, from)` – looks for `item` starting from index `from`, returns `true` if found.

Usually, these methods are used with only one argument: the `item` to search. By default, the search is from the beginning. For instance:

```
let arr = [1, 0, false];

alert( arr.indexOf(0) ); // 1
alert( arr.indexOf(false) ); // 2
alert( arr.indexOf(null) ); // -1

alert( arr.includes(1) ); // true
```

Please note that `indexOf` uses the strict equality `===` for comparison. So, if we look for `false`, it finds exactly `false` and not the zero.

If we want to check if `item` exists in the array and don't need the index, then `arr.includes` is preferred.

The method `arr.lastIndexOf` is the same as `indexOf`, but looks for from right to left.

```
let fruits = ['Apple', 'Orange', 'Apple']

alert( fruits.indexOf('Apple') ); // 0 (first Apple)
alert( fruits.lastIndexOf('Apple') ); // 2 (last Apple)
```

i The `includes` method handles `NaN` correctly

A minor, but noteworthy feature of `includes` is that it correctly handles `NaN`, unlike `indexOf`:

```
const arr = [NaN];
alert( arr.indexOf(NaN) ); // -1 (wrong, should be 0)
alert( arr.includes(NaN) ); // true (correct)
```

That's because `includes` was added to JavaScript much later and uses the more up-to-date comparison algorithm internally.

find and findIndex/findLastIndex

Imagine we have an array of objects. How do we find an object with a specific condition?

Here the `arr.find(fn)` method comes in handy.

The syntax is:

```
let result = arr.find(function(item, index, array) {
  // if true is returned, item is returned and iteration is stopped
  // for falsy scenario returns undefined
});
```

The function is called for elements of the array, one after another:

- `item` is the element.
- `index` is its index.
- `array` is the array itself.

If it returns `true`, the search is stopped, the `item` is returned. If nothing is found, `undefined` is returned.

For example, we have an array of users, each with the fields `id` and `name`. Let's find the one with `id == 1`:

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"}
];

let user = users.find(item => item.id == 1);
```

```
alert(user.name); // John
```

In real life, arrays of objects are a common thing, so the `find` method is very useful.

Note that in the example we provide to `find` the function `item => item.id == 1` with one argument. That's typical, other arguments of this function are rarely used.

The [arr.findIndex](#) method has the same syntax but returns the index where the element was found instead of the element itself. The value of `-1` is returned if nothing is found.

The [arr.findLastIndex](#) method is like `findIndex`, but searches from right to left, similar to `lastIndexOf`.

Here's an example:

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"},
  {id: 4, name: "John"}
];

// Find the index of the first John
alert(users.findIndex(user => user.name == 'John')); // 0

// Find the index of the last John
alert(users.findLastIndex(user => user.name == 'John')); // 3
```

filter

The `find` method looks for a single (first) element that makes the function return `true`.

If there may be many, we can use [arr.filter\(fn\)](#).

The syntax is similar to `find`, but `filter` returns an array of all matching elements:

```
let results = arr.filter(function(item, index, array) {
  // if true item is pushed to results and the iteration continues
  // returns empty array if nothing found
});
```

For instance:

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"}
];

// returns array of the first two users
let someUsers = users.filter(item => item.id < 3);

alert(someUsers.length); // 2
```

Transform an array

Let's move on to methods that transform and reorder an array.

map

The [arr.map](#) method is one of the most useful and often used.

It calls the function for each element of the array and returns the array of results.

The syntax is:

```
let result = arr.map(function(item, index, array) {
  // returns the new value instead of item
});
```

For instance, here we transform each element into its length:

```
let lengths = ["Bilbo", "Gandalf", "Nazgul"].map(item => item.length);
alert(lengths); // 5,7,6
```

sort(fn)

The call to `arr.sort()` sorts the array *in place*, changing its element order.

It also returns the sorted array, but the returned value is usually ignored, as `arr` itself is modified.

For instance:

```
let arr = [ 1, 2, 15 ];

// the method reorders the content of arr
arr.sort();

alert( arr ); // 1, 15, 2
```

Did you notice anything strange in the outcome?

The order became `1, 15, 2`. Incorrect. But why?

The items are sorted as strings by default.

Literally, all elements are converted to strings for comparisons. For strings, lexicographic ordering is applied and indeed `"2" > "15"`.

To use our own sorting order, we need to supply a function as the argument of `arr.sort()`.

The function should compare two arbitrary values and return:

```
function compare(a, b) {
  if (a > b) return 1; // if the first value is greater than the second
  if (a == b) return 0; // if values are equal
  if (a < b) return -1; // if the first value is less than the second
}
```

For instance, to sort as numbers:

```
function compareNumeric(a, b) {
  if (a > b) return 1;
  if (a == b) return 0;
  if (a < b) return -1;
}

let arr = [ 1, 2, 15 ];

arr.sort(compareNumeric);

alert(arr); // 1, 2, 15
```

Now it works as intended.

Let's step aside and think about what's happening. The `arr` can be an array of anything, right? It may contain numbers or strings or objects or whatever. We have a set of *some items*. To sort it, we need an *ordering function* that knows how to compare its elements. The default is a string order.

The `arr.sort(fn)` method implements a generic sorting algorithm. We don't need to care how it internally works (an optimized [quicksort](#) or [Timsort](#) most of the time). It will walk the array, compare its elements using the provided function and reorder them, all we need is to provide the `fn` which does the comparison.

By the way, if we ever want to know which elements are compared – nothing prevents us from alerting them:

```
[1, -2, 15, 2, 0, 8].sort(function(a, b) {
  alert( a + " <> " + b );
```

```
return a - b;
});
```

The algorithm may compare an element with multiple others in the process, but it tries to make as few comparisons as possible.

i A comparison function may return any number

Actually, a comparison function is only required to return a positive number to say “greater” and a negative number to say “less”.

That allows to write shorter functions:

```
let arr = [ 1, 2, 15 ];

arr.sort(function(a, b) { return a - b; });

alert(arr); // 1, 2, 15
```

i Arrow functions for the best

Remember [arrow functions](#)? We can use them here for neater sorting:

```
arr.sort( (a, b) => a - b );
```

This works exactly the same as the longer version above.

i Use `localeCompare` for strings

Remember [strings](#) comparison algorithm? It compares letters by their codes by default.

For many alphabets, it's better to use `str.localeCompare` method to correctly sort letters, such as `Ö`.

For example, let's sort a few countries in German:

```
let countries = ['Österreich', 'Andorra', 'Vietnam'];

alert( countries.sort( (a, b) => a > b ? 1 : -1) ); // Andorra, Vietnam, Österreich (wrong)

alert( countries.sort( (a, b) => a.localeCompare(b) ) ); // Andorra,Österreich,Vietnam (correct!)
```

reverse

The method `arr.reverse` [↗](#) reverses the order of elements in `arr`.

For instance:

```
let arr = [1, 2, 3, 4, 5];
arr.reverse();

alert( arr ); // 5,4,3,2,1
```

It also returns the array `arr` after the reversal.

split and join

Here's the situation from real life. We are writing a messaging app, and the person enters the comma-delimited list of receivers: `John, Pete, Mary`. But for us an array of names would be much more comfortable than a single string. How to get it?

The `str.split(delim)` [↗](#) method does exactly that. It splits the string into an array by the given delimiter `delim`.

In the example below, we split by a comma followed by a space:

```
let names = 'Bilbo, Gandalf, Nazgul';

let arr = names.split(', ');

for (let name of arr) {
  alert( `A message to ${name}.` ); // A message to Bilbo (and other names)
}
```

The `split` method has an optional second numeric argument – a limit on the array length. If it is provided, then the extra elements are ignored. In practice it is rarely used though:

```
let arr = 'Bilbo, Gandalf, Nazgul, Saruman'.split(', ', 2);

alert(arr); // Bilbo, Gandalf
```

Split into letters

The call to `split(s)` with an empty `s` would split the string into an array of letters:

```
let str = "test";

alert( str.split('') ); // t,e,s,t
```

The call `arr.join(glue)` [↗](#) does the reverse to `split`. It creates a string of `arr` items joined by `glue` between them.

For instance:

```
let arr = ['Bilbo', 'Gandalf', 'Nazgul'];

let str = arr.join(';'); // glue the array into a string using ;

alert( str ); // Bilbo;Gandalf;Nazgul
```

reduce/reduceRight

When we need to iterate over an array – we can use `forEach`, `for` or `for..of`.

When we need to iterate and return the data for each element – we can use `map`.

The methods `arr.reduce` [↗](#) and `arr.reduceRight` [↗](#) also belong to that breed, but are a little bit more intricate. They are used to calculate a single value based on the array.

The syntax is:

```
let value = arr.reduce(function(accumulator, item, index, array) {
  // ...
}, [initial]);
```

The function is applied to all array elements one after another and “carries on” its result to the next call.

Arguments:

- `accumulator` – is the result of the previous function call, equals `initial` the first time (if `initial` is provided).
- `item` – is the current array item.
- `index` – is its position.
- `array` – is the array.

As the function is applied, the result of the previous function call is passed to the next one as the first argument.

So, the first argument is essentially the accumulator that stores the combined result of all previous executions. And at the end, it becomes the result of `reduce`.

Sounds complicated?

The easiest way to grasp that is by example.

Here we get a sum of an array in one line:

```
let arr = [1, 2, 3, 4, 5];

let result = arr.reduce((sum, current) => sum + current, 0);

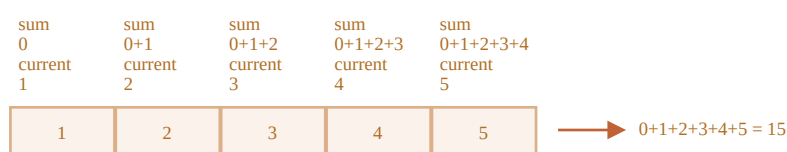
alert(result); // 15
```

The function passed to `reduce` uses only 2 arguments, that's typically enough.

Let's see the details of what's going on.

1. On the first run, `sum` is the `initial` value (the last argument of `reduce`), equals `0`, and `current` is the first array element, equals `1`. So the function result is `1`.
2. On the second run, `sum = 1`, we add the second array element (`2`) to it and return.
3. On the 3rd run, `sum = 3` and we add one more element to it, and so on...

The calculation flow:



Or in the form of a table, where each row represents a function call on the next array element:

	sum	current	result
the first call	0	1	1
the second call	1	2	3
the third call	3	3	6
the fourth call	6	4	10
the fifth call	10	5	15

Here we can clearly see how the result of the previous call becomes the first argument of the next one.

We also can omit the initial value:

```
let arr = [1, 2, 3, 4, 5];

// removed initial value from reduce (no 0)
let result = arr.reduce((sum, current) => sum + current);

alert( result ); // 15
```

The result is the same. That's because if there's no initial, then `reduce` takes the first element of the array as the initial value and starts the iteration from the 2nd element.

The calculation table is the same as above, minus the first row.

But such use requires an extreme care. If the array is empty, then `reduce` call without initial value gives an error.

Here's an example:

```
let arr = [];

// Error: Reduce of empty array with no initial value
// if the initial value existed, reduce would return it for the empty arr.
arr.reduce((sum, current) => sum + current);
```

So it's advised to always specify the initial value.

The method `arr.reduceRight` does the same but goes from right to left.

importance: 4

The [factorial](#) of a natural number is a number multiplied by "number minus one", then by "number minus two", and so on till 1. The factorial of n is denoted as $n!$

We can write a definition of factorial like this:

```
n! = n * (n - 1) * (n - 2) * ... * 1
```

Values of factorials for different n :

```
1! = 1
2! = 2 * 1 = 2
3! = 3 * 2 * 1 = 6
4! = 4 * 3 * 2 * 1 = 24
5! = 5 * 4 * 3 * 2 * 1 = 120
```

The task is to write a function `factorial(n)` that calculates $n!$ using recursive calls.

```
alert( factorial(5) ); // 120
```

P.S. Hint: $n!$ can be written as $n * (n-1)!$ For instance: $3! = 3*2! = 3*2*1! = 6$

[To solution](#)

Fibonacci numbers

importance: 5

The sequence of [Fibonacci numbers](#) has the formula $F_n = F_{n-1} + F_{n-2}$. In other words, the next number is a sum of the two preceding ones.

First two numbers are 1, then 2(1+1), then 3(1+2), 5(2+3) and so on: 1, 1, 2, 3, 5, 8, 13, 21...

Fibonacci numbers are related to the [Golden ratio](#) and many natural phenomena around us.

Write a function `fib(n)` that returns the n -th Fibonacci number.

An example of work:

```
function fib(n) { /* your code */ }

alert(fib(3)); // 2
alert(fib(7)); // 13
alert(fib(77)); // 5527939700884757
```

P.S. The function should be fast. The call to `fib(77)` should take no more than a fraction of a second.

[To solution](#)

Output a single-linked list

importance: 5

Let's say we have a single-linked list (as described in the chapter [Recursion and stack](#)):

```
let list = {
  value: 1,
  next: {
    value: 2,
    next: {
      value: 3,
      next: {
        value: 4,
        next: null
      }
    }
  }
}
```

```
}  
}  
};
```

Write a function `printList(list)` that outputs list items one-by-one.

Make two variants of the solution: using a loop and using recursion.

What's better: with recursion or without it?

[To solution](#)

Output a single-linked list in the reverse order

importance: 5

Output a single-linked list from the previous task [Output a single-linked list](#) in the reverse order.

Make two solutions: using a loop and using a recursion.

[To solution](#)

Rest parameters and spread syntax

Many JavaScript built-in functions support an arbitrary number of arguments.

For instance:

- `Math.max(arg1, arg2, ..., argN)` – returns the greatest of the arguments.
- `Object.assign(dest, src1, ..., srcN)` – copies properties from `src1..N` into `dest`.
- ...and so on.

In this chapter we'll learn how to do the same. And also, how to pass arrays to such functions as parameters.

Rest parameters ...

A function can be called with any number of arguments, no matter how it is defined.

Like here:

```
function sum(a, b) {  
  return a + b;  
}  
  
alert( sum(1, 2, 3, 4, 5) );
```

There will be no error because of “excessive” arguments. But of course in the result only the first two will be counted, so the result in the code above is `3`.

The rest of the parameters can be included in the function definition by using three dots `...` followed by the name of the array that will contain them. The dots literally mean “gather the remaining parameters into an array”.

For instance, to gather all arguments into array `args`:

```
function sumAll(...args) { // args is the name for the array  
  let sum = 0;  
  
  for (let arg of args) sum += arg;  
  
  return sum;  
}  
  
alert( sumAll(1) ); // 1  
alert( sumAll(1, 2) ); // 3  
alert( sumAll(1, 2, 3) ); // 6
```

We can choose to get the first parameters as variables, and gather only the rest.

Here the first two arguments go into variables and the rest go into `titles` array:

```
function showName(firstName, lastName, ...titles) {
  alert( firstName + ' ' + lastName ); // Julius Caesar

  // the rest go into titles array
  // i.e. titles = ["Consul", "Imperator"]
  alert( titles[0] ); // Consul
  alert( titles[1] ); // Imperator
  alert( titles.length ); // 2
}

showName("Julius", "Caesar", "Consul", "Imperator");
```

⚠ The rest parameters must be at the end

The rest parameters gather all remaining arguments, so the following does not make sense and causes an error:

```
function f(arg1, ...rest, arg2) { // arg2 after ...rest ?!
  // error
}
```

The `...rest` must always be last.

The “arguments” variable

There is also a special array-like object named `arguments` that contains all arguments by their index.

For instance:

```
function showName() {
  alert( arguments.length );
  alert( arguments[0] );
  alert( arguments[1] );

  // it's iterable
  // for(let arg of arguments) alert(arg);
}

// shows: 2, Julius, Caesar
showName("Julius", "Caesar");

// shows: 1, Ilya, undefined (no second argument)
showName("Ilya");
```

In old times, rest parameters did not exist in the language, and using `arguments` was the only way to get all arguments of the function. And it still works, we can find it in the old code.

But the downside is that although `arguments` is both array-like and iterable, it's not an array. It does not support array methods, so we can't call `arguments.map(...)` for example.

Also, it always contains all arguments. We can't capture them partially, like we did with rest parameters.

So when we need these features, then rest parameters are preferred.

i Arrow functions do not have "arguments"

If we access the `arguments` object from an arrow function, it takes them from the outer "normal" function.

Here's an example:

```
function f() {  
  let showArg = () => alert(arguments[0]);  
  showArg();  
}  
  
f(1); // 1
```

As we remember, arrow functions don't have their own `this`. Now we know they don't have the special `arguments` object either.

Spread syntax

We've just seen how to get an array from the list of parameters.

But sometimes we need to do exactly the reverse.

For instance, there's a built-in function `Math.max` [↗](#) that returns the greatest number from a list:

```
alert( Math.max(3, 5, 1) ); // 5
```

Now let's say we have an array `[3, 5, 1]`. How do we call `Math.max` with it?

Passing it "as is" won't work, because `Math.max` expects a list of numeric arguments, not a single array:

```
let arr = [3, 5, 1];  
  
alert( Math.max(arr) ); // NaN
```

And surely we can't manually list items in the code `Math.max(arr[0], arr[1], arr[2])`, because we may be unsure how many there are. As our script executes, there could be a lot, or there could be none. And that would get ugly.

Spread syntax to the rescue! It looks similar to rest parameters, also using `...`, but does quite the opposite.

When `...arr` is used in the function call, it "expands" an iterable object `arr` into the list of arguments.

For `Math.max`:

```
let arr = [3, 5, 1];  
  
alert( Math.max(...arr) ); // 5 (spread turns array into a list of arguments)
```

We also can pass multiple iterables this way:

```
let arr1 = [1, -2, 3, 4];  
let arr2 = [8, 3, -8, 1];  
  
alert( Math.max(...arr1, ...arr2) ); // 8
```

We can even combine the spread syntax with normal values:

```
let arr1 = [1, -2, 3, 4];  
let arr2 = [8, 3, -8, 1];  
  
alert( Math.max(1, ...arr1, 2, ...arr2, 25) ); // 25
```

Also, the spread syntax can be used to merge arrays:

```
let arr = [3, 5, 1];
let arr2 = [8, 9, 15];

let merged = [0, ...arr, 2, ...arr2];

alert(merged); // 0,3,5,1,2,8,9,15 (0, then arr, then 2, then arr2)
```

In the examples above we used an array to demonstrate the spread syntax, but any iterable will do.

For instance, here we use the spread syntax to turn the string into array of characters:

```
let str = "Hello";

alert( [...str] ); // H,e,l,l,o
```

The spread syntax internally uses iterators to gather elements, the same way as `for...of` does.

So, for a string, `for...of` returns characters and `...str` becomes `"H", "e", "l", "l", "o"`. The list of characters is passed to array initializer `[...str]`.

For this particular task we could also use `Array.from`, because it converts an iterable (like a string) into an array:

```
let str = "Hello";

// Array.from converts an iterable into an array
alert( Array.from(str) ); // H,e,l,l,o
```

The result is the same as `[...str]`.

But there's a subtle difference between `Array.from(obj)` and `[...obj]`:

- `Array.from` operates on both array-likes and iterables.
- The spread syntax works only with iterables.

So, for the task of turning something into an array, `Array.from` tends to be more universal.

Copy an array/object

Remember when we talked about `Object.assign()` [in the past](#)?

It is possible to do the same thing with the spread syntax.

```
let arr = [1, 2, 3];

let arrCopy = [...arr]; // spread the array into a list of parameters
                        // then put the result into a new array

// do the arrays have the same contents?
alert(JSON.stringify(arr) === JSON.stringify(arrCopy)); // true

// are the arrays equal?
alert(arr === arrCopy); // false (not same reference)

// modifying our initial array does not modify the copy:
arr.push(4);
alert(arr); // 1, 2, 3, 4
alert(arrCopy); // 1, 2, 3
```

Note that it is possible to do the same thing to make a copy of an object:

```
let obj = { a: 1, b: 2, c: 3 };

let objCopy = { ...obj }; // spread the object into a list of parameters
                        // then return the result in a new object

// do the objects have the same contents?
alert(JSON.stringify(obj) === JSON.stringify(objCopy)); // true
```

```
// are the objects equal?
alert(obj === objCopy); // false (not same reference)

// modifying our initial object does not modify the copy:
obj.d = 4;
alert(JSON.stringify(obj)); // {"a":1,"b":2,"c":3,"d":4}
alert(JSON.stringify(objCopy)); // {"a":1,"b":2,"c":3}
```

This way of copying an object is much shorter than `let objCopy = Object.assign({}, obj)` or for an array `let arrCopy = Object.assign([], arr)` so we prefer to use it whenever we can.

Summary

When we see `"..."` in the code, it is either rest parameters or the spread syntax.

There's an easy way to distinguish between them:

- When `...` is at the end of function parameters, it's "rest parameters" and gathers the rest of the list of arguments into an array.
- When `...` occurs in a function call or alike, it's called a "spread syntax" and expands an array into a list.

Use patterns:

- Rest parameters are used to create functions that accept any number of arguments.
- The spread syntax is used to pass an array to functions that normally require a list of many arguments.

Together they help to travel between a list and an array of parameters with ease.

All arguments of a function call are also available in "old-style" `arguments`: array-like iterable object.

Variable scope, closure

JavaScript is a very function-oriented language. It gives us a lot of freedom. A function can be created at any moment, passed as an argument to another function, and then called from a totally different place of code later.

We already know that a function can access variables outside of it ("outer" variables).

But what happens if outer variables change since a function is created? Will the function get newer values or the old ones?

And what if a function is passed along as an argument and called from another place of code, will it get access to outer variables at the new place?

Let's expand our knowledge to understand these scenarios and more complex ones.

We'll talk about `let/const` variables here

In JavaScript, there are 3 ways to declare a variable: `let`, `const` (the modern ones), and `var` (the remnant of the past).

- In this article we'll use `let` variables in examples.
- Variables, declared with `const`, behave the same, so this article is about `const` too.
- The old `var` has some notable differences, they will be covered in the article [The old "var"](#).

Code blocks

If a variable is declared inside a code block `{...}`, it's only visible inside that block.

For example:

```
{
  // do some job with local variables that should not be seen outside

  let message = "Hello"; // only visible in this block

  alert(message); // Hello
}
```

```
alert(message); // Error: message is not defined
```

We can use this to isolate a piece of code that does its own task, with variables that only belong to it:

```
{
  // show message
  let message = "Hello";
  alert(message);
}

{
  // show another message
  let message = "Goodbye";
  alert(message);
}
```

i There'd be an error without blocks

Please note, without separate blocks there would be an error, if we use `let` with the existing variable name:

```
// show message
let message = "Hello";
alert(message);

// show another message
let message = "Goodbye"; // Error: variable already declared
alert(message);
```

For `if`, `for`, `while` and so on, variables declared in `{...}` are also only visible inside:

```
if (true) {
  let phrase = "Hello!";

  alert(phrase); // Hello!
}

alert(phrase); // Error, no such variable!
```

Here, after `if` finishes, the `alert` below won't see the `phrase`, hence the error.

That's great, as it allows us to create block-local variables, specific to an `if` branch.

The similar thing holds true for `for` and `while` loops:

```
for (let i = 0; i < 3; i++) {
  // the variable i is only visible inside this for
  alert(i); // 0, then 1, then 2
}

alert(i); // Error, no such variable
```

Visually, `let i` is outside of `{...}`. But the `for` construct is special here: the variable, declared inside it, is considered a part of the block.

Nested functions

A function is called "nested" when it is created inside another function.

It is easily possible to do this with JavaScript.

We can use it to organize our code, like this:

```
function sayHiBye(firstName, lastName) {
```

```
// helper nested function to use below
function getFullName() {
    return firstName + " " + lastName;
}

alert( "Hello, " + getFullName() );
alert( "Bye, " + getFullName() );

}
```

Here the *nested* function `getFullName()` is made for convenience. It can access the outer variables and so can return the full name. Nested functions are quite common in JavaScript.

What's much more interesting, a nested function can be returned: either as a property of a new object or as a result by itself. It can then be used somewhere else. No matter where, it still has access to the same outer variables.

Below, `makeCounter` creates the “counter” function that returns the next number on each invocation:

```
function makeCounter() {
    let count = 0;

    return function() {
        return count++;
    };
}

let counter = makeCounter();

alert( counter() ); // 0
alert( counter() ); // 1
alert( counter() ); // 2
```

Despite being simple, slightly modified variants of that code have practical uses, for instance, as a [random number generator](#) [↗](#) to generate random values for automated tests.

How does this work? If we create multiple counters, will they be independent? What's going on with the variables here?

Understanding such things is great for the overall knowledge of JavaScript and beneficial for more complex scenarios. So let's go a bit in-depth.

Lexical Environment

Here be dragons!

The in-depth technical explanation lies ahead.

As far as I'd like to avoid low-level language details, any understanding without them would be lacking and incomplete, so get ready.

For clarity, the explanation is split into multiple steps.

Step 1. Variables

In JavaScript, every running function, code block `{...}`, and the script as a whole have an internal (hidden) associated object known as the *Lexical Environment*.

The Lexical Environment object consists of two parts:

1. *Environment Record* – an object that stores all local variables as its properties (and some other information like the value of `this`).
2. A reference to the *outer lexical environment*, the one associated with the outer code.

A “variable” is just a property of the special internal object, `Environment Record`. “To get or change a variable” means “to get or change a property of that object”.

In this simple code without functions, there is only one Lexical Environment:

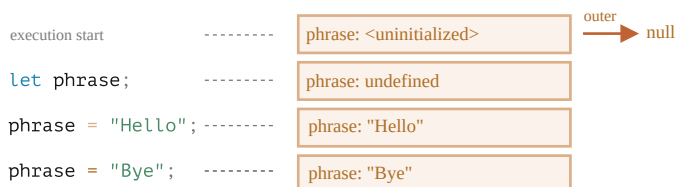


This is the so-called *global* Lexical Environment, associated with the whole script.

On the picture above, the rectangle means Environment Record (variable store) and the arrow means the outer reference. The global Lexical Environment has no outer reference, that's why the arrow points to `null`.

As the code starts executing and goes on, the Lexical Environment changes.

Here's a little bit longer code:



Rectangles on the right-hand side demonstrate how the global Lexical Environment changes during the execution:

1. When the script starts, the Lexical Environment is pre-populated with all declared variables.
 - Initially, they are in the "Uninitialized" state. That's a special internal state, it means that the engine knows about the variable, but it cannot be referenced until it has been declared with `let`. It's almost the same as if the variable didn't exist.
2. Then `let phrase` definition appears. There's no assignment yet, so its value is `undefined`. We can use the variable from this point forward.
3. `phrase` is assigned a value.
4. `phrase` changes the value.

Everything looks simple for now, right?

- A variable is a property of a special internal object, associated with the currently executing block/function/script.
- Working with variables is actually working with the properties of that object.

i Lexical Environment is a specification object

"Lexical Environment" is a specification object: it only exists "theoretically" in the [language specification](#) to describe how things work. We can't get this object in our code and manipulate it directly.

JavaScript engines also may optimize it, discard variables that are unused to save memory and perform other internal tricks, as long as the visible behavior remains as described.

Step 2. Function Declarations

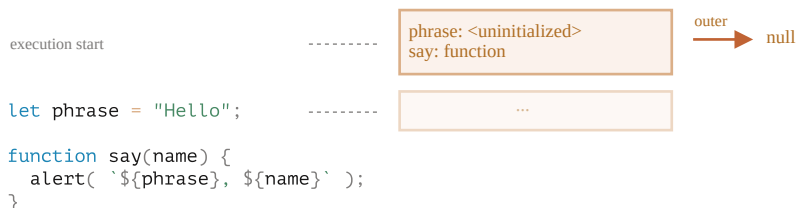
A function is also a value, like a variable.

The difference is that a Function Declaration is instantly fully initialized.

When a Lexical Environment is created, a Function Declaration immediately becomes a ready-to-use function (unlike `let`, that is unusable till the declaration).

That's why we can use a function, declared as Function Declaration, even before the declaration itself.

For example, here's the initial state of the global Lexical Environment when we add a function:

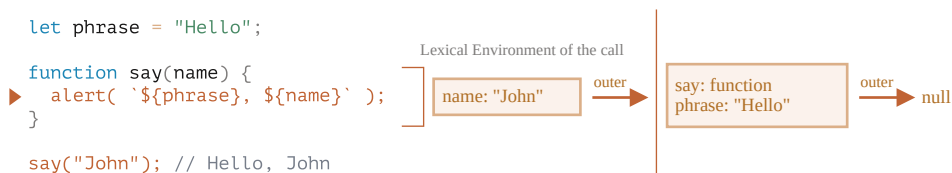


Naturally, this behavior only applies to Function Declarations, not Function Expressions where we assign a function to a variable, such as `let say = function(name)...`

Step 3. Inner and outer Lexical Environment

When a function runs, at the beginning of the call, a new Lexical Environment is created automatically to store local variables and parameters of the call.

For instance, for `say("John")`, it looks like this (the execution is at the line, labelled with an arrow):



During the function call we have two Lexical Environments: the inner one (for the function call) and the outer one (global):

- The inner Lexical Environment corresponds to the current execution of `say`. It has a single property: `name`, the function argument. We called `say("John")`, so the value of the `name` is `"John"`.
- The outer Lexical Environment is the global Lexical Environment. It has the `phrase` variable and the function itself.

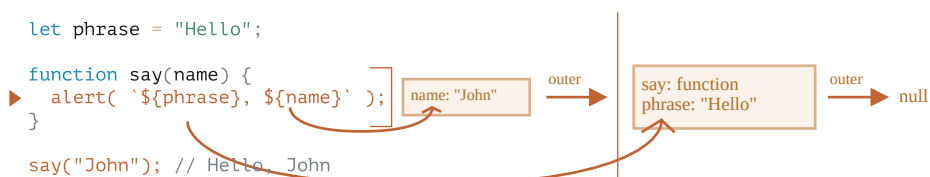
The inner Lexical Environment has a reference to the `outer` one.

When the code wants to access a variable – the inner Lexical Environment is searched first, then the outer one, then the more outer one and so on until the global one.

If a variable is not found anywhere, that's an error in strict mode (without `use strict`, an assignment to a non-existing variable creates a new global variable, for compatibility with old code).

In this example the search proceeds as follows:

- For the `name` variable, the `alert` inside `say` finds it immediately in the inner Lexical Environment.
- When it wants to access `phrase`, then there is no `phrase` locally, so it follows the reference to the outer Lexical Environment and finds it there.



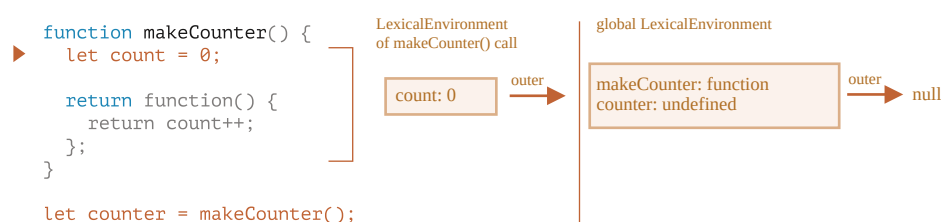
Step 4. Returning a function

Let's return to the `makeCounter` example.

```
function makeCounter() {  
  let count = 0;  
  
  return function() {  
    return count++;  
  };  
}  
  
let counter = makeCounter();
```

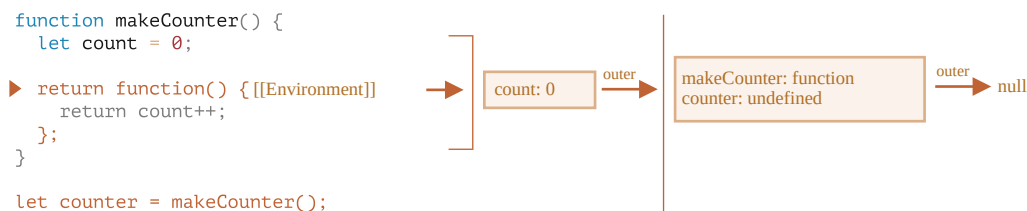
At the beginning of each `makeCounter()` call, a new Lexical Environment object is created, to store variables for this `makeCounter` run.

So we have two nested Lexical Environments, just like in the example above:



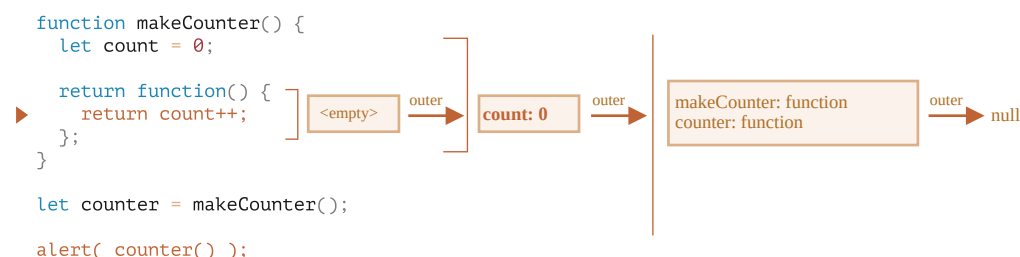
What's different is that, during the execution of `makeCounter()`, a tiny nested function is created of only one line: `return count++`. We don't run it yet, only create.

All functions remember the Lexical Environment in which they were made. Technically, there's no magic here: all functions have the hidden property named `[[Environment]]`, that keeps the reference to the Lexical Environment where the function was created:



So, `counter.[[Environment]]` has the reference to `{count: 0}` Lexical Environment. That's how the function remembers where it was created, no matter where it's called. The `[[Environment]]` reference is set once and forever at function creation time.

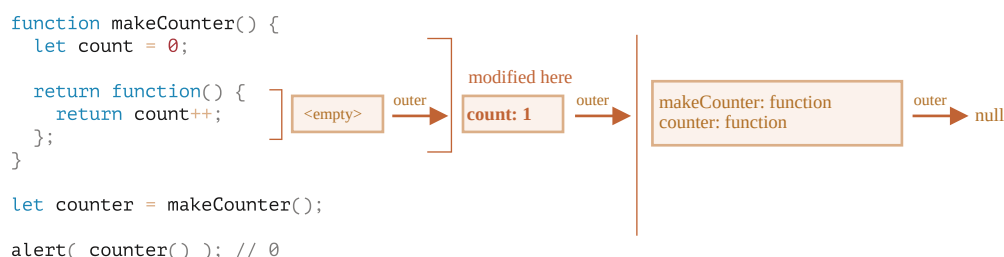
Later, when `counter()` is called, a new Lexical Environment is created for the call, and its outer Lexical Environment reference is taken from `counter.[[Environment]]`:



Now when the code inside `counter()` looks for `count` variable, it first searches its own Lexical Environment (empty, as there are no local variables there), then the Lexical Environment of the outer `makeCounter()` call, where it finds and changes it.

A variable is updated in the Lexical Environment where it lives.

Here's the state after the execution:



If we call `counter()` multiple times, the `count` variable will be increased to `2`, `3` and so on, at the same place.

i Closure

There is a general programming term "closure", that developers generally should know.

A [closure](#) is a function that remembers its outer variables and can access them. In some languages, that's not possible, or a function should be written in a special way to make it happen. But as explained above, in JavaScript, all functions are naturally closures (there is only one exception, to be covered in [The "new Function" syntax](#)).

That is: they automatically remember where they were created using a hidden `[[Environment]]` property, and then their code can access outer variables.

When on an interview, a frontend developer gets a question about "what's a closure?", a valid answer would be a definition of the closure and an explanation that all functions in JavaScript are closures, and maybe a few more words about technical details: the `[[Environment]]` property and how Lexical Environments work.

Here's the code for it (uses the debounce decorator from the [Lodash library](#)):

```
let f = _.debounce(alert, 1000);

f("a");
setTimeout( () => f("b"), 200);
setTimeout( () => f("c"), 500);
// debounced function waits 1000ms after the last call and then runs: alert("c")
```

Now a practical example. Let's say, the user types something, and we'd like to send a request to the server when the input is finished.

There's no point in sending the request for every character typed. Instead we'd like to wait, and then process the whole result.

In a web-browser, we can setup an event handler – a function that's called on every change of an input field. Normally, an event handler is called very often, for every typed key. But if we **debounce** it by 1000ms, then it will be only called once, after 1000ms after the last input.

So, **debounce** is a great way to process a sequence of events: be it a sequence of key presses, mouse movements or something else.

It waits the given time after the last call, and then runs its function, that can process the result.

The task is to implement **debounce** decorator.

Hint: that's just a few lines if you think about it :)

[Open a sandbox with tests.](#)

[To solution](#)

Throttle decorator

importance: 5

Create a “throttling” decorator **throttle(f, ms)** – that returns a wrapper.

When it's called multiple times, it passes the call to **f** at maximum once per **ms** milliseconds.

Compared to the debounce decorator, the behavior is completely different:

- **debounce** runs the function once after the “cooldown” period. Good for processing the final result.
- **throttle** runs it not more often than given **ms** time. Good for regular updates that shouldn't be very often.

In other words, **throttle** is like a secretary that accepts phone calls, but bothers the boss (calls the actual **f**) not more often than once per **ms** milliseconds.

Let's check the real-life application to better understand that requirement and to see where it comes from.

For instance, we want to track mouse movements.

In a browser we can setup a function to run at every mouse movement and get the pointer location as it moves. During an active mouse usage, this function usually runs very frequently, can be something like 100 times per second (every 10 ms).

We'd like to update some information on the web-page when the pointer moves.

...But updating function **update()** is too heavy to do it on every micro-movement. There is also no sense in updating more often than once per 100ms.

So we'll wrap it into the decorator: use **throttle(update, 100)** as the function to run on each mouse move instead of the original **update()**. The decorator will be called often, but forward the call to **update()** at maximum once per 100ms.

Visually, it will look like this:

1. For the first mouse movement the decorated variant immediately passes the call to **update**. That's important, the user sees our reaction to their move immediately.
2. Then as the mouse moves on, until **100ms** nothing happens. The decorated variant ignores calls.

3. At the end of `100ms` – one more `update` happens with the last coordinates.
4. Then, finally, the mouse stops somewhere. The decorated variant waits until `100ms` expire and then runs `update` with last coordinates. So, quite important, the final mouse coordinates are processed.

A code example:

```
function f(a) {
  console.log(a);
}

// f1000 passes calls to f at maximum once per 1000 ms
let f1000 = throttle(f, 1000);

f1000(1); // shows 1
f1000(2); // (throttling, 1000ms not out yet)
f1000(3); // (throttling, 1000ms not out yet)

// when 1000 ms time out...
// ...outputs 3, intermediate value 2 was ignored
```

P.S. Arguments and the context `this` passed to `f1000` should be passed to the original `f`.

[Open a sandbox with tests.](#)

[To solution](#)

Function binding

When passing object methods as callbacks, for instance to `setTimeout`, there's a known problem: "losing `this`".

In this chapter we'll see the ways to fix it.

Losing "this"

We've already seen examples of losing `this`. Once a method is passed somewhere separately from the object – `this` is lost.

Here's how it may happen with `setTimeout`:

```
let user = {
  firstName: "John",
  sayHi() {
    alert(`Hello, ${this.firstName}!`);
  }
};

setTimeout(user.sayHi, 1000); // Hello, undefined!
```

As we can see, the output shows not "John" as `this.firstName`, but `undefined`!

That's because `setTimeout` got the function `user.sayHi`, separately from the object. The last line can be rewritten as:

```
let f = user.sayHi;
setTimeout(f, 1000); // lost user context
```

The method `setTimeout` in-browser is a little special: it sets `this=window` for the function call (for Node.js, `this` becomes the timer object, but doesn't really matter here). So for `this.firstName` it tries to get `window.firstName`, which does not exist. In other similar cases, usually `this` just becomes `undefined`.

The task is quite typical – we want to pass an object method somewhere else (here – to the scheduler) where it will be called. How to make sure that it will be called in the right context?

Solution 1: a wrapper

The simplest solution is to use a wrapping function:

```
let user = {
  firstName: "John",
  sayHi() {
    alert(`Hello, ${this.firstName}!`);
  }
};

setTimeout(function() {
  user.sayHi(); // Hello, John!
}, 1000);
```

Now it works, because it receives `user` from the outer lexical environment, and then calls the method normally.

The same, but shorter:

```
setTimeout(() => user.sayHi(), 1000); // Hello, John!
```

Looks fine, but a slight vulnerability appears in our code structure.

What if before `setTimeout` triggers (there's one second delay!) `user` changes value? Then, suddenly, it will call the wrong object!

```
let user = {
  firstName: "John",
  sayHi() {
    alert(`Hello, ${this.firstName}!`);
  }
};

setTimeout(() => user.sayHi(), 1000);

// ...the value of user changes within 1 second
user = {
  sayHi() { alert("Another user in setTimeout!"); }
};

// Another user in setTimeout!
```

The next solution guarantees that such thing won't happen.

Solution 2: bind

Functions provide a built-in method `bind` [↗](#) that allows to fix `this`.

The basic syntax is:

```
// more complex syntax will come a little later
let boundFunc = func.bind(context);
```

The result of `func.bind(context)` is a special function-like “exotic object”, that is callable as function and transparently passes the call to `func` setting `this=context`.

In other words, calling `boundFunc` is like `func` with fixed `this`.

For instance, here `funcUser` passes a call to `func` with `this=user`:

```
let user = {
  firstName: "John"
};

function func() {
  alert(this.firstName);
}
```

```

}

let funcUser = func.bind(user);
funcUser(); // John

```

Here `func.bind(user)` is a “bound variant” of `func`, with fixed `this=user`.

All arguments are passed to the original `func` “as is”, for instance:

```

let user = {
  firstName: "John"
};

function func(phrase) {
  alert(phrase + ', ' + this.firstName);
}

// bind this to user
let funcUser = func.bind(user);

funcUser("Hello"); // Hello, John (argument "Hello" is passed, and this=user)

```

Now let's try with an object method:

```

let user = {
  firstName: "John",
  sayHi() {
    alert(`Hello, ${this.firstName}!`);
  }
};

let sayHi = user.sayHi.bind(user); // (*)

// can run it without an object
sayHi(); // Hello, John!

setTimeout(sayHi, 1000); // Hello, John!

// even if the value of user changes within 1 second
// sayHi uses the pre-bound value which is reference to the old user object
user = {
  sayHi() { alert("Another user in setTimeout!"); }
};

```

In the line `(*)` we take the method `user.sayHi` and bind it to `user`. The `sayHi` is a “bound” function, that can be called alone or passed to `setTimeout` – doesn't matter, the context will be right.

Here we can see that arguments are passed “as is”, only `this` is fixed by `bind`:

```

let user = {
  firstName: "John",
  say(phrase) {
    alert(`${phrase}, ${this.firstName}!`);
  }
};

let say = user.say.bind(user);

say("Hello"); // Hello, John! ("Hello" argument is passed to say)
say("Bye"); // Bye, John! ("Bye" is passed to say)

```

Convenience method: `bindAll`

If an object has many methods and we plan to actively pass it around, then we could bind them all in a loop:

```
for (let key in user) {
  if (typeof user[key] == 'function') {
    user[key] = user[key].bind(user);
  }
}
```

JavaScript libraries also provide functions for convenient mass binding , e.g. `_.bindAll(object, methodNames)` [↗](#) in `lodash`.

Partial functions

Until now we have only been talking about binding `this` . Let's take it a step further.

We can bind not only `this` , but also arguments. That's rarely done, but sometimes can be handy.

The full syntax of `bind` :

```
let bound = func.bind(context, [arg1], [arg2], ...);
```

It allows to bind context as `this` and starting arguments of the function.

For instance, we have a multiplication function `mul(a, b)` :

```
function mul(a, b) {
  return a * b;
}
```

Let's use `bind` to create a function `double` on its base:

```
function mul(a, b) {
  return a * b;
}

let double = mul.bind(null, 2);

alert( double(3) ); // = mul(2, 3) = 6
alert( double(4) ); // = mul(2, 4) = 8
alert( double(5) ); // = mul(2, 5) = 10
```

The call to `mul.bind(null, 2)` creates a new function `double` that passes calls to `mul` , fixing `null` as the context and `2` as the first argument. Further arguments are passed "as is".

That's called [partial function application](#) [↗](#) – we create a new function by fixing some parameters of the existing one.

Please note that we actually don't use `this` here. But `bind` requires it, so we must put in something like `null` .

The function `triple` in the code below triples the value:

```
function mul(a, b) {
  return a * b;
}

let triple = mul.bind(null, 3);

alert( triple(3) ); // = mul(3, 3) = 9
alert( triple(4) ); // = mul(3, 4) = 12
alert( triple(5) ); // = mul(3, 5) = 15
```

Why do we usually make a partial function?

The benefit is that we can create an independent function with a readable name (`double`, `triple`). We can use it and not provide the first argument every time as it's fixed with `bind`.

In other cases, partial application is useful when we have a very generic function and want a less universal variant of it for convenience.

For instance, we have a function `send(from, to, text)`. Then, inside a `user` object we may want to use a partial variant of it: `sendTo(to, text)` that sends from the current user.

Going partial without context

What if we'd like to fix some arguments, but not the context `this`? For example, for an object method.

The native `bind` does not allow that. We can't just omit the context and jump to arguments.

Fortunately, a function `partial` for binding only arguments can be easily implemented.

Like this:

```
function partial(func, ...argsBound) {
  return function(...args) { // (*)
    return func.call(this, ...argsBound, ...args);
  }
}

// Usage:
let user = {
  firstName: "John",
  say(time, phrase) {
    alert(`[${time}] ${this.firstName}: ${phrase}!`);
  }
};

// add a partial method with fixed time
user.sayNow = partial(user.say, new Date().getHours() + ':' + new Date().getMinutes());

user.sayNow("Hello");
// Something like:
// [10:00] John: Hello!
```

The result of `partial(func[, arg1, arg2...])` call is a wrapper `(*)` that calls `func` with:

- Same `this` as it gets (for `user.sayNow` call it's `user`)
- Then gives it `...argsBound` – arguments from the `partial` call (`"10:00"`)
- Then gives it `...args` – arguments given to the wrapper (`"Hello"`)

So easy to do it with the spread syntax, right?

Also there's a ready `_.partial` [↗](#) implementation from lodash library.

Summary

Method `func.bind(context, ...args)` returns a "bound variant" of function `func` that fixes the context `this` and first arguments if given.

Usually we apply `bind` to fix `this` for an object method, so that we can pass it somewhere. For example, to `setTimeout`.

When we fix some arguments of an existing function, the resulting (less universal) function is called *partially applied* or *partial*.

Partials are convenient when we don't want to repeat the same argument over and over again. Like if we have a `send(from, to)` function, and `from` should always be the same for our task, we can get a partial and go on with it.